

딥러닝 알고리즘 기초

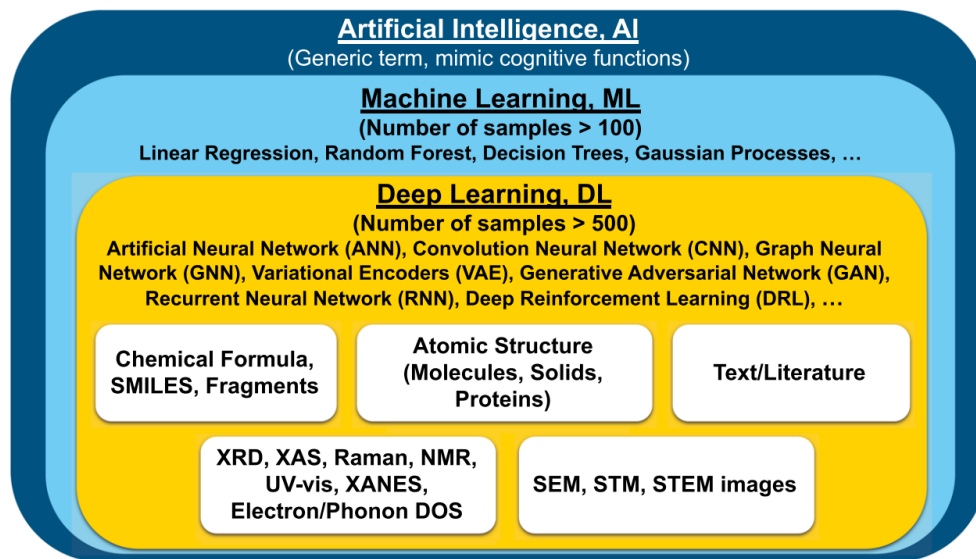
이예리

한국화학연구원 디지털화학연구센터

yealee@krict.re.kr

목차

- 딥러닝 소개: 퍼셉트론과 인공 신경망 (약 15분)
- 학습 과정: 순전파 · 비용함수 · 역전파 · optimizer (약 35분)
- 예제와 일반화: 와인 분류, 과적합, regularization (약 20분)
- 신경망 구조와 그래프 신경망: CNN, GNN, GCN (약 20분)
- 소재 GNN: CGCNN에서 범용 퍼텐셜 · 생성 모델까지 (약 30분)



Deep Learning

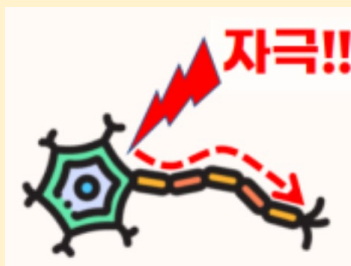
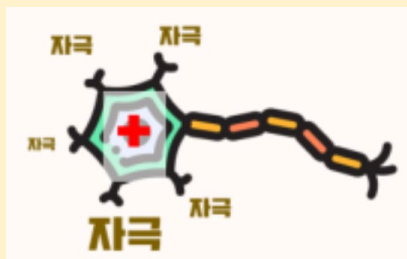
딥러닝이란?

인공 신경망을 기반으로 학습하는 방법

인공 신경망 (ANN: Artificial Neural Network)

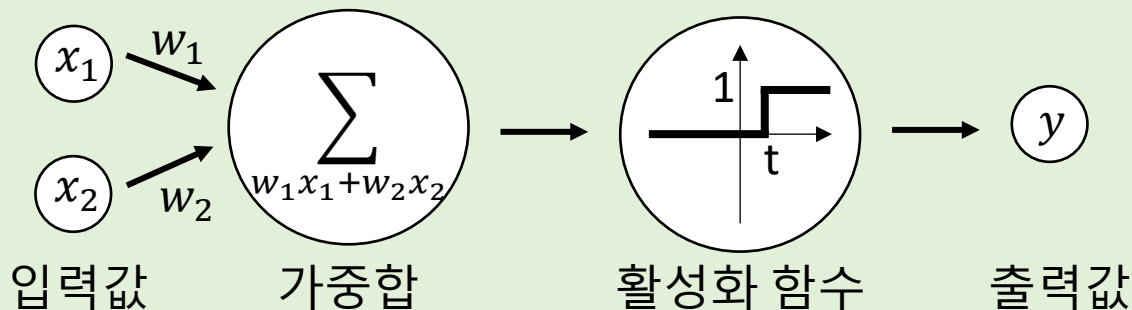
사람의 신경계인 뉴런(Neuron)이 신경망으로 연결된 형태를 모방하여 만든 수학적 모델

Neuron



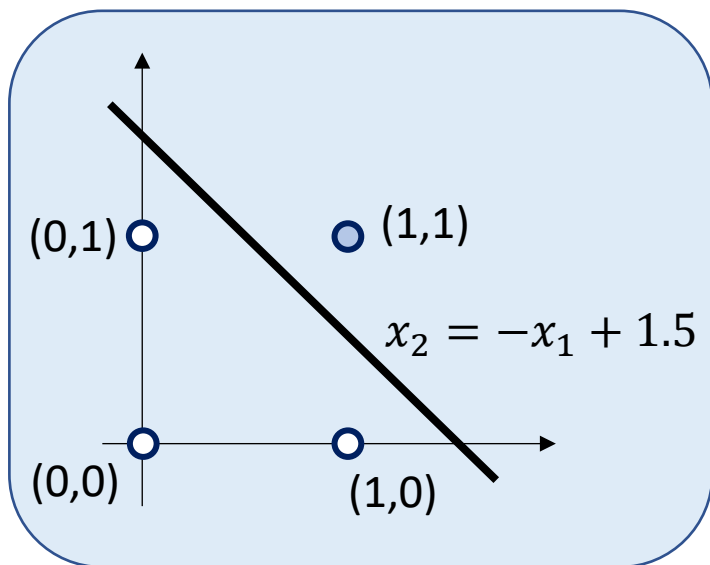
https://blog.naver.com/walk_along/222864824998

Perceptron



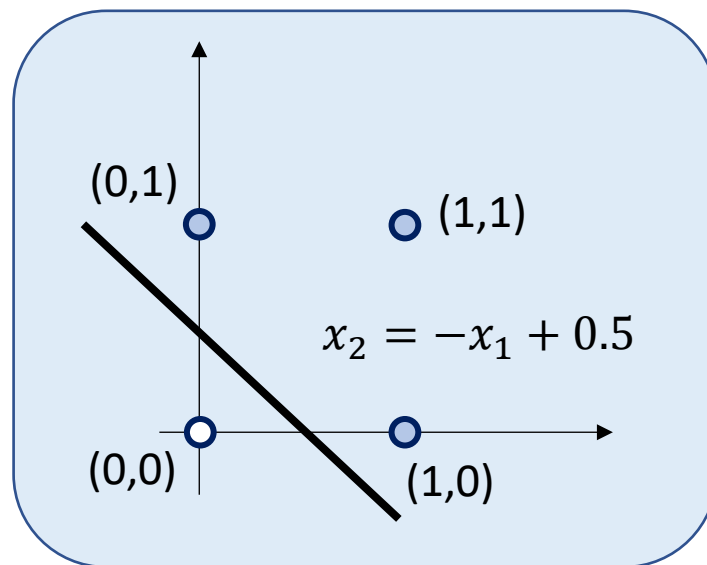
Perceptron

$$p_1(x_1, x_2) = x_1 + x_2 - 1.5$$



AND	x_1	x_2	p_1	y
(0,0)	0	0	-1.5	0
(1,0)	1	0	-0.5	0
(0,1)	0	1	-0.5	0
(1,1)	1	1	0.5	1

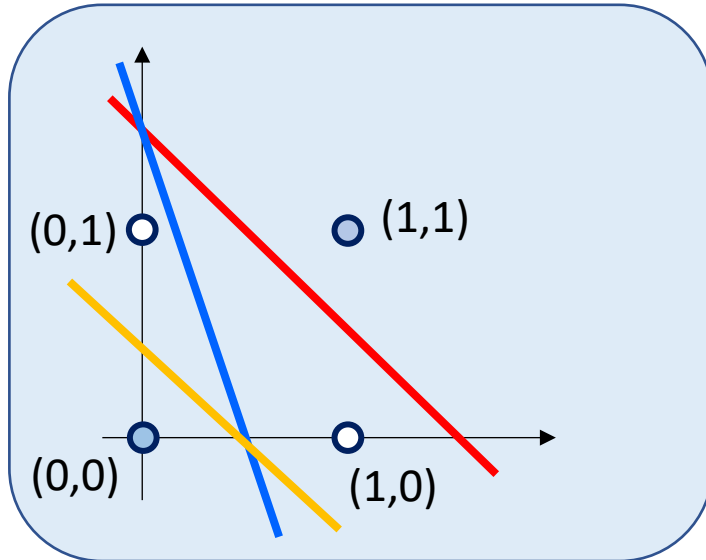
$$p_2(x_1, x_2) = x_1 + x_2 - 0.5$$



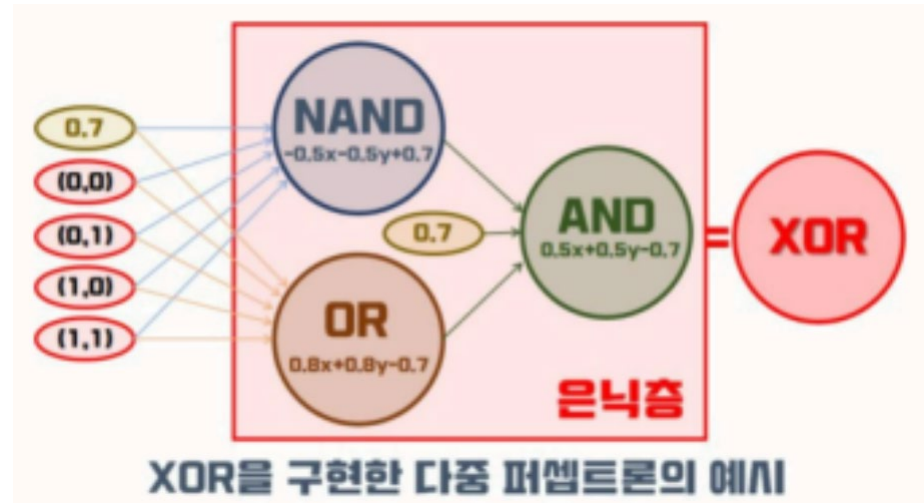
OR	x_1	x_2	p_2	y
(0,0)	0	0	-0.5	0
(1,0)	1	0	0.5	1
(0,1)	0	1	0.5	1
(1,1)	1	1	1.5	1

Multi-layer perceptron

$$p_1(x_1, x_2) = ?$$

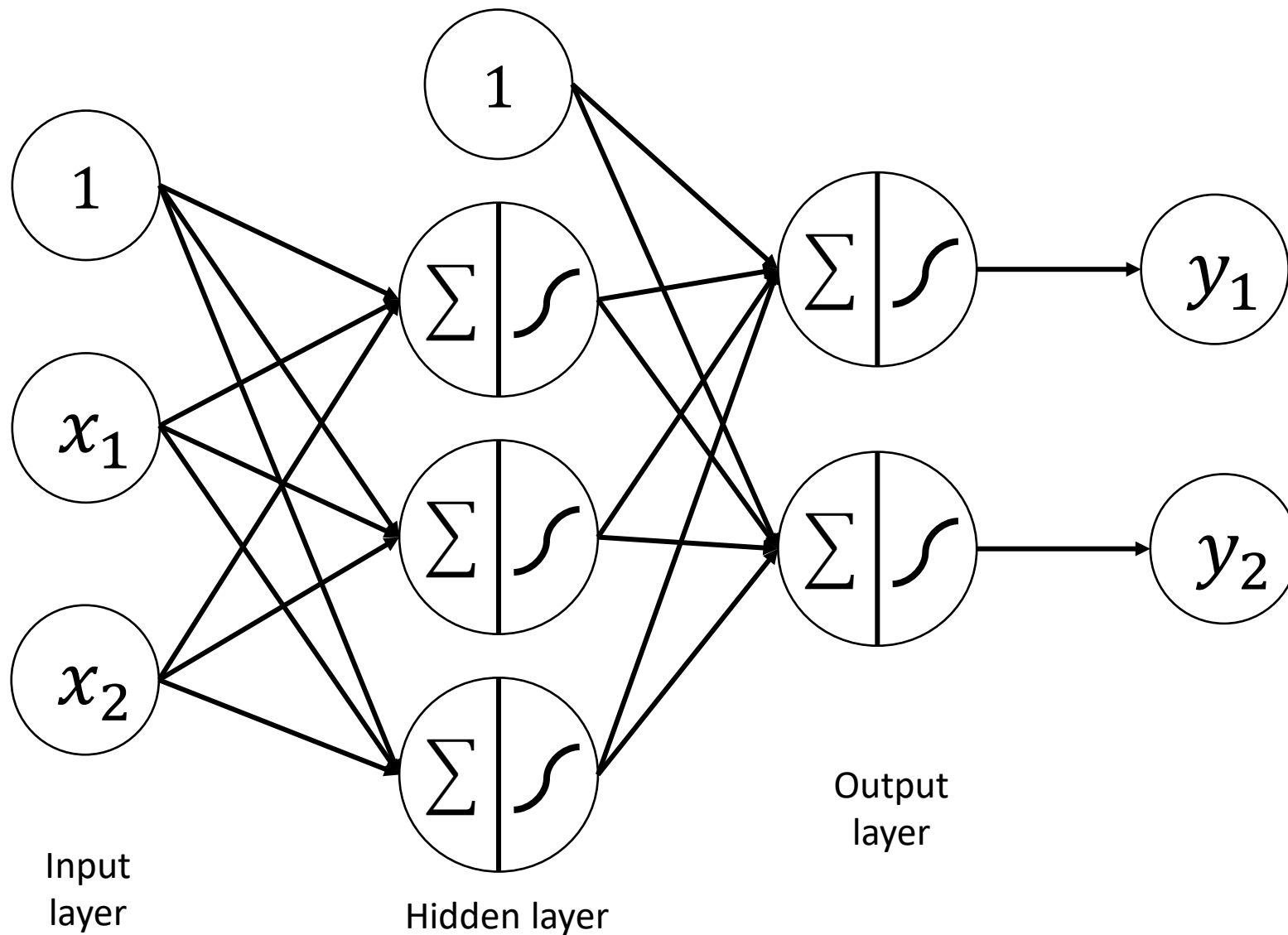


XOR	x_1	x_2	p_1	y
(0,0)	0	0	?	0
(1,0)	1	0	?	1
(0,1)	0	1	?	1
(1,1)	1	1	?	0

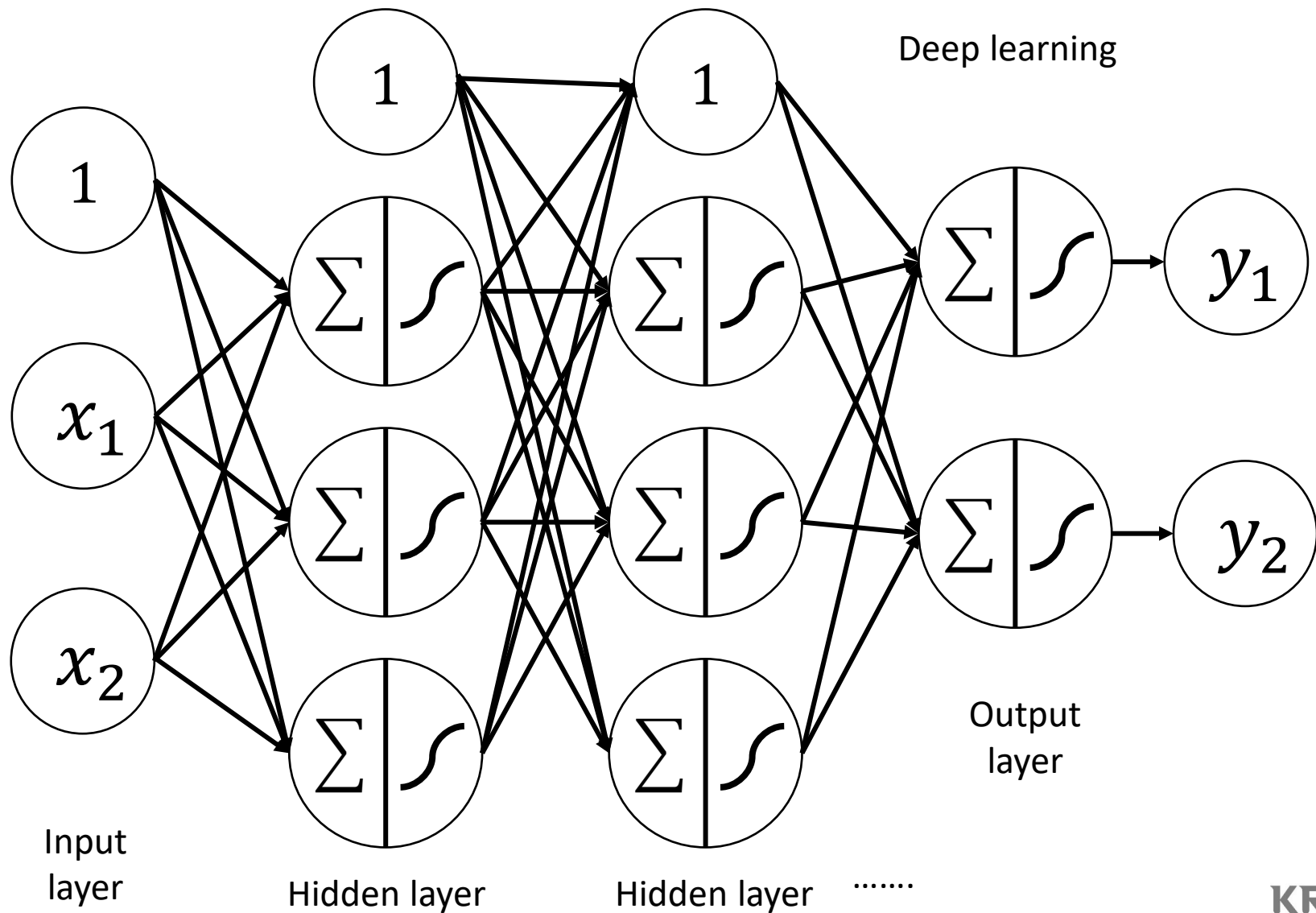


https://blog.naver.com/walk_along/222864824998

Artificial neural network

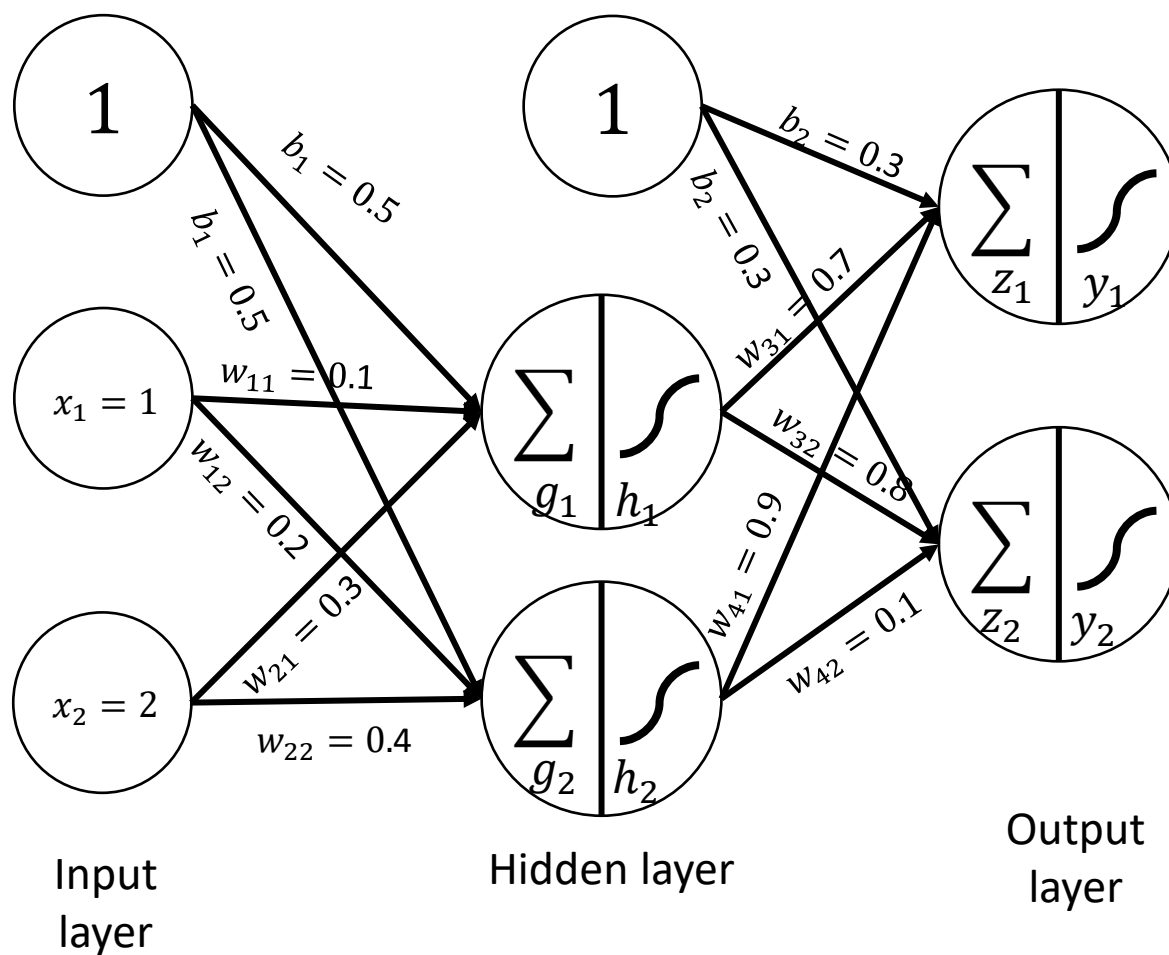


Artificial neural network



Learning process

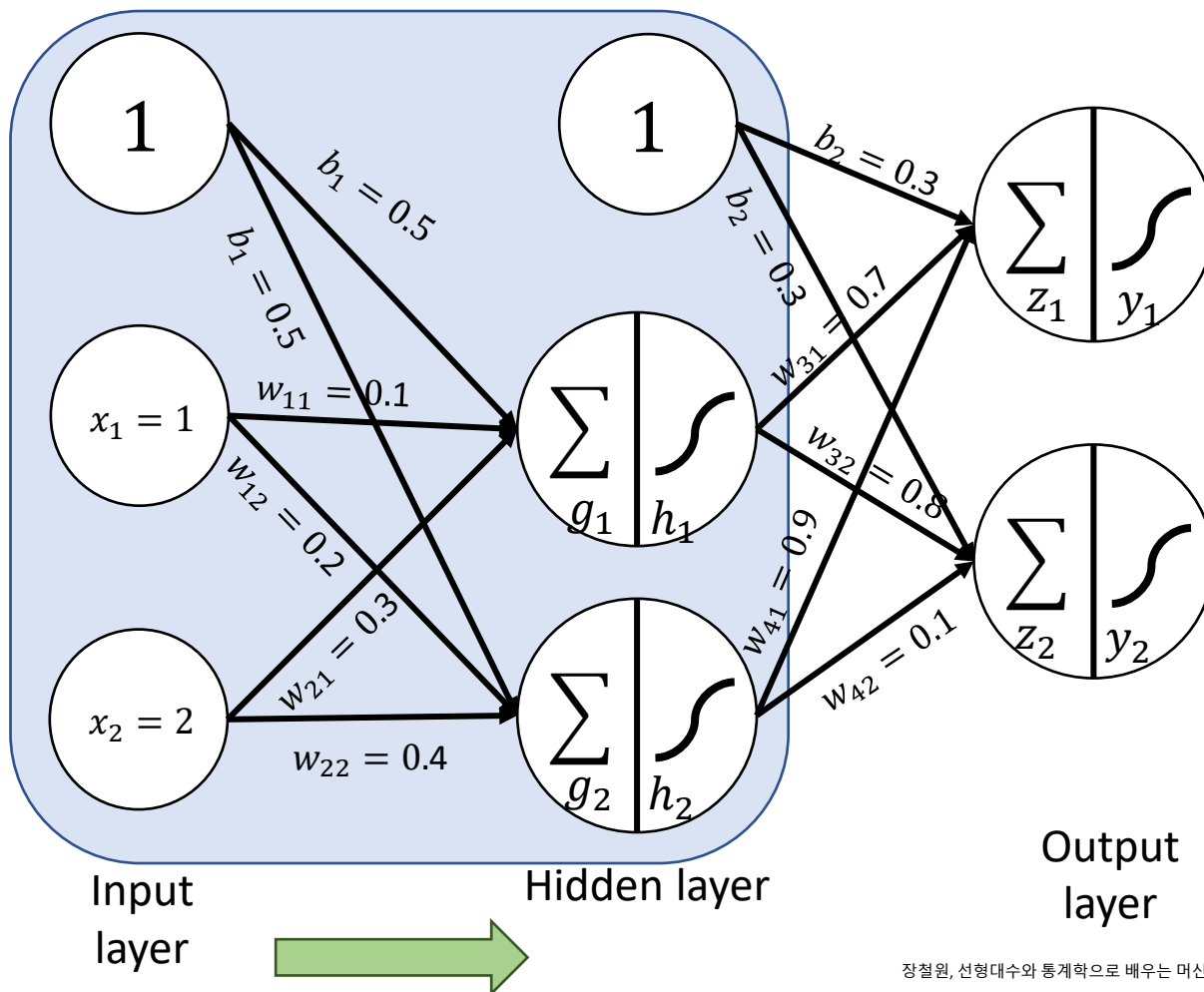
1. Initialize weight w



Learning process

2. Making prediction - forward propagation

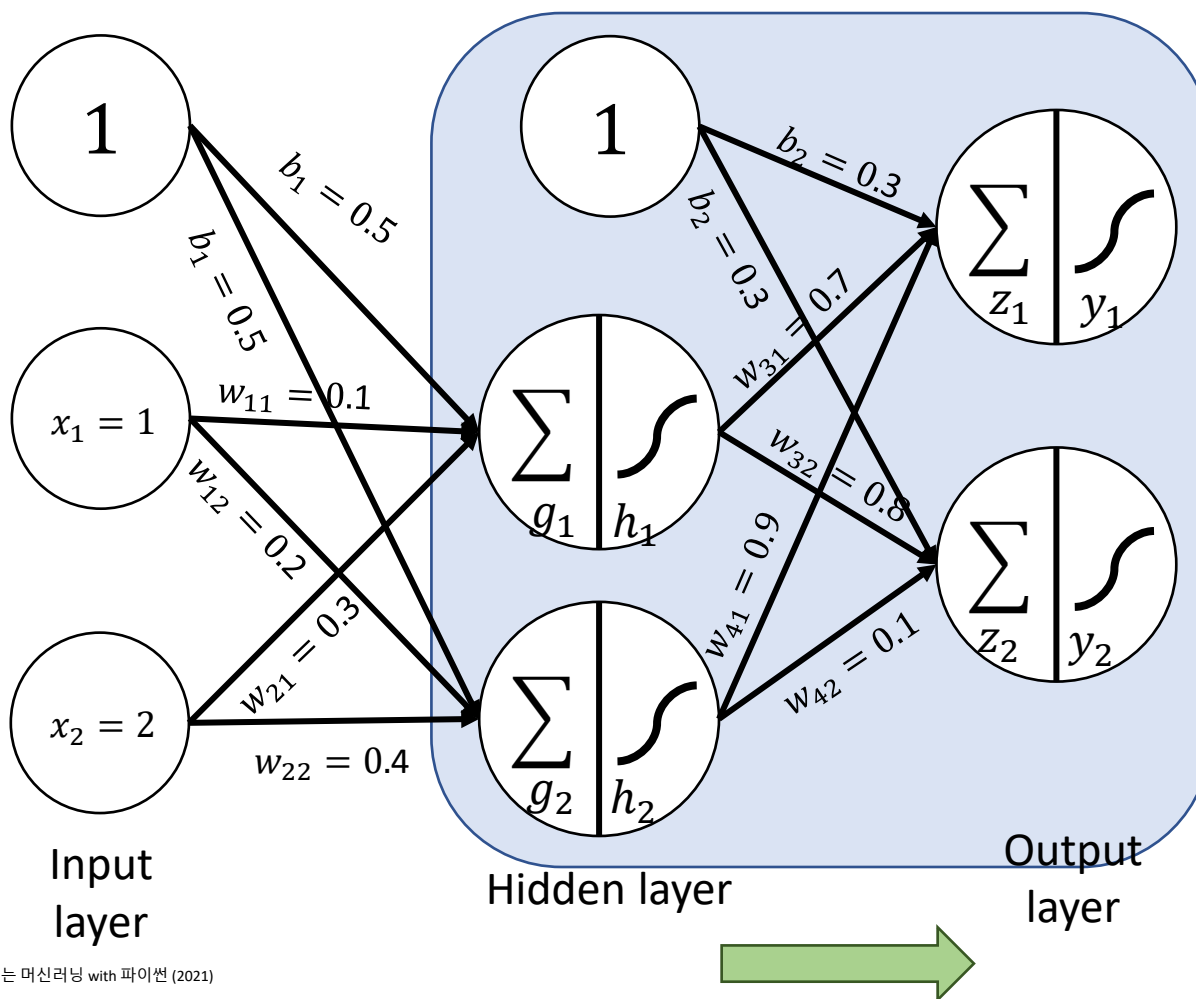
$$W_1^T \mathbf{x} + b_1 = \mathbf{g} \rightarrow \mathbf{h} = \phi(\mathbf{g})$$



Learning process

2. Making prediction : forward propagation

$$W_2^T \mathbf{h} + b_2 = \mathbf{z} \rightarrow \mathbf{y} = \phi(\mathbf{z})$$



Forward propagation 계산 예

입력 $x = (1, 2)$, 활성화 함수 $\sigma = \text{sigmoid}$

Hidden layer: $g = W_1^T x + b_1$, $h = \sigma(g)$

1행: $0.1 \times 1 + 0.3 \times 2 + 0.5 = 1.2$

$$g = \begin{bmatrix} \overset{W_1^T}{\mathbf{0.1} & \mathbf{0.3}} \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ 1 \\ 2 \end{bmatrix} + \begin{bmatrix} \mathbf{b}_1 \\ 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} \mathbf{g} \\ \mathbf{1.2} \\ 1.5 \end{bmatrix} \rightarrow \mathbf{h} = \sigma(\mathbf{g}) = \begin{bmatrix} \mathbf{h} \\ \mathbf{0.769} \\ 0.818 \end{bmatrix}$$

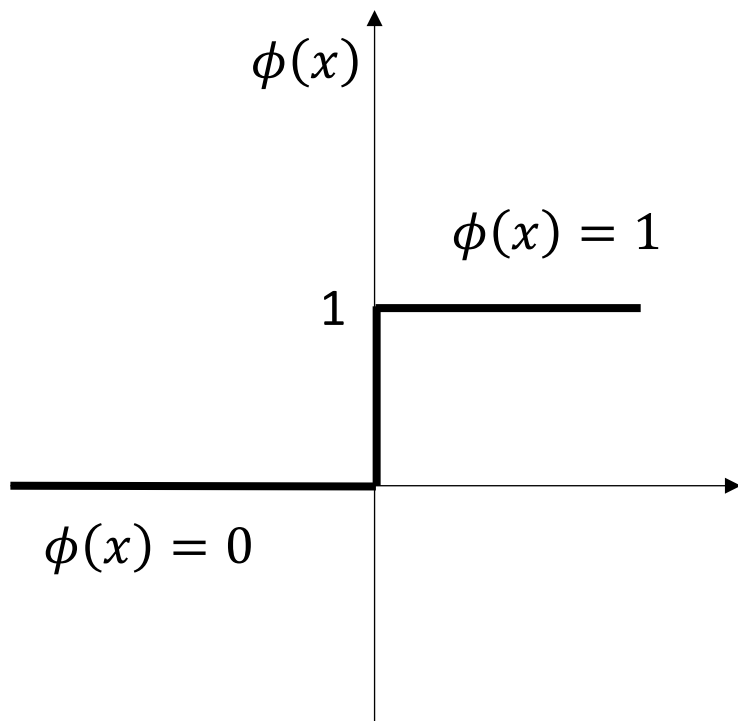
Output layer: $z = W_2^T h + b_2$, $y = \sigma(z)$

1행: $0.7 \times 0.769 + 0.9 \times 0.818 + 0.3 = 1.574$

$$\mathbf{z} = \begin{bmatrix} \overset{W_2^T}{\mathbf{0.7} & \mathbf{0.9}} \\ 0.8 & 0.1 \end{bmatrix} \begin{bmatrix} \mathbf{h} \\ 0.769 \\ 0.818 \end{bmatrix} + \begin{bmatrix} \mathbf{b}_2 \\ 0.3 \\ 0.3 \end{bmatrix} = \begin{bmatrix} \mathbf{z} \\ \mathbf{1.574} \\ 0.997 \end{bmatrix} \rightarrow \mathbf{y} = \sigma(\mathbf{z}) = \begin{bmatrix} \mathbf{y} \\ \mathbf{0.828} \\ 0.730 \end{bmatrix}$$

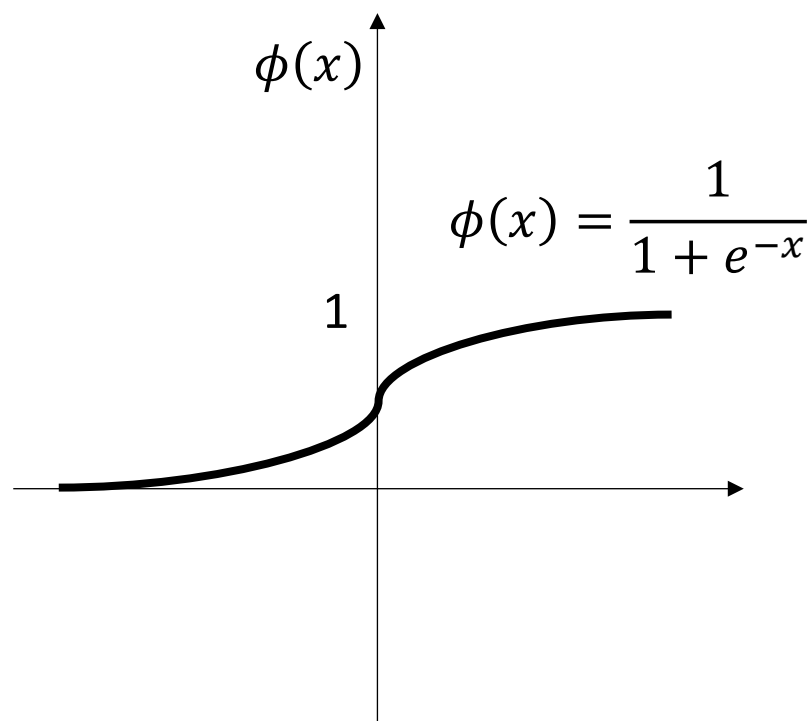
- 가중치가 임의의 초기값이므로 출력 $y = (0.828, 0.730)$ 도 아직 의미 없는 값
- 정답 t 와 비교해서 가중치를 고쳐 나가는 과정이 "학습" $\rightarrow$ cost function

Activation function 1



Step function

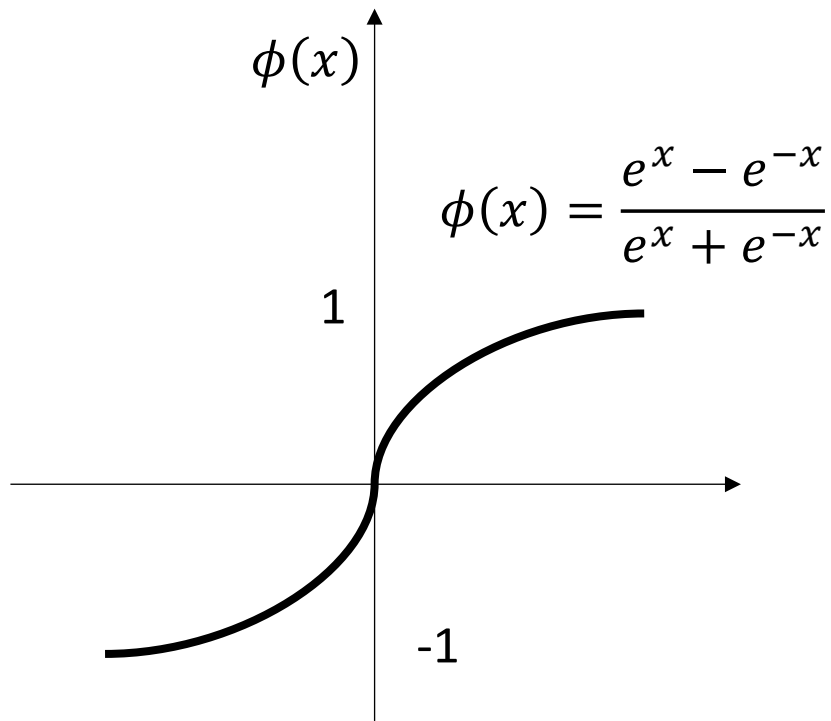
$$\phi(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$



Sigmoid function

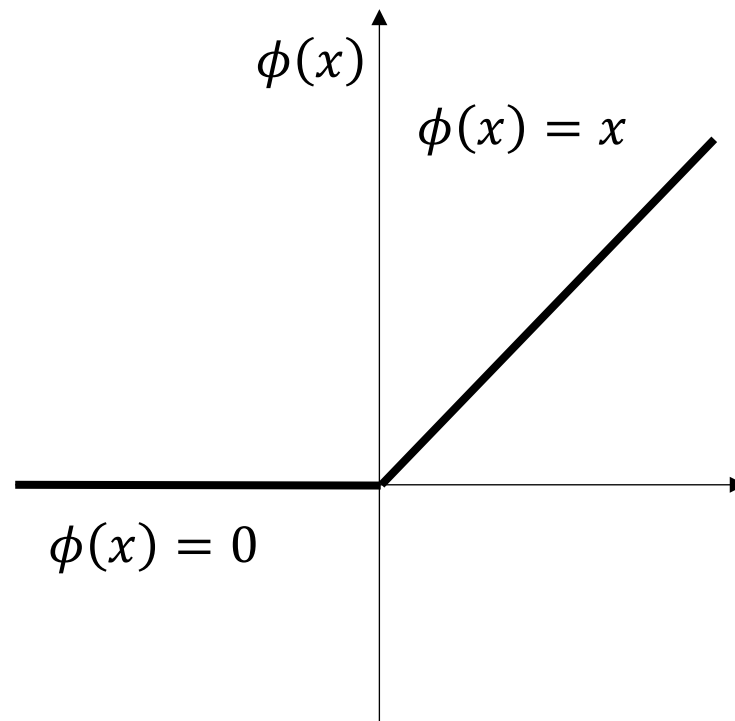
$$\phi(x) = \frac{1}{1 + e^{-x}}$$

Activation function 2



Hyperbolic tangent
function

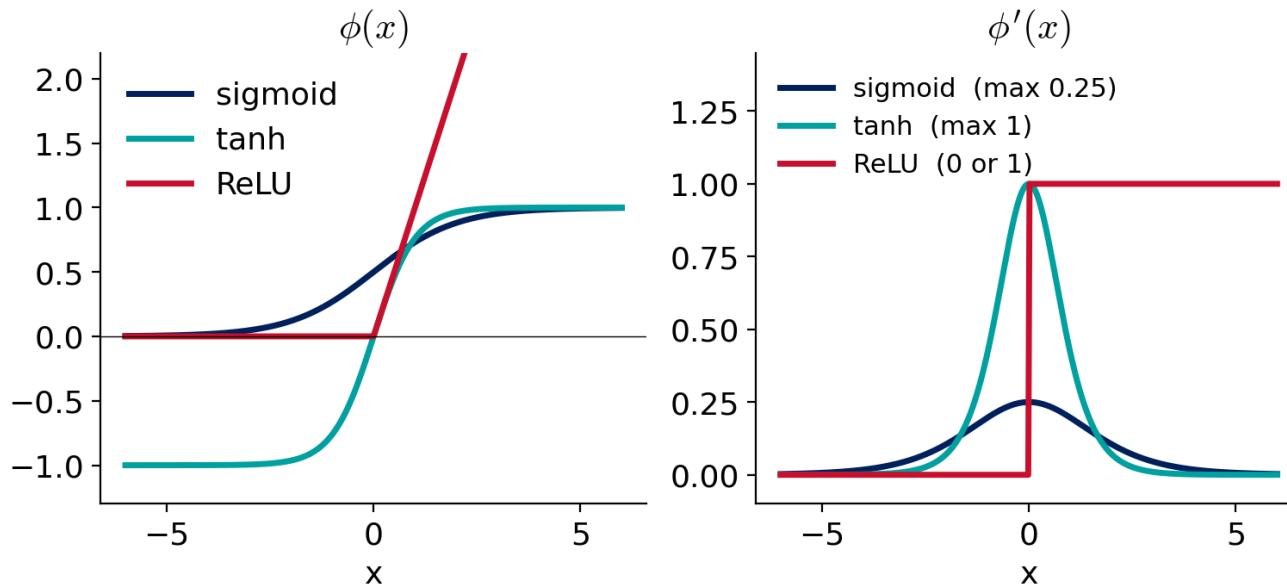
$$\phi(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Rectified Linear Unit
(ReLU) function

$$\phi(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

Activation function 3



- **Sigmoid, tanh:** $|x|$ 가 크면 기울기 ≈ 0 . 층이 깊어질수록 기울기가 곱해져 사라짐 (vanishing gradient)
- **ReLU:** $x > 0$ 에서 기울기 1. 깊은 신경망 은닉층의 기본 선택 (변형: Leaky ReLU, SiLU, softplus)
- **출력층:** 회귀 $\rightarrow$ 항등함수, 이진 분류 $\rightarrow$ sigmoid, 다중 분류 $\rightarrow$ softmax
- 에너지를 미분해 힘을 구하는 원자간 퍼텐셜 모델은 매끄러운 함수(SiLU, softplus)를 주로 사용

Learning process

3. Cost function

Comparison between target ($\mathbf{t}$) and prediction ($\mathbf{y}$)

Minimize the cost function for prediction

ex) $C = \frac{1}{2}[(y_1 - t_1)^2 + (y_2 - t_2)^2]$: mean squared error

$$C = \frac{1}{2}[|y_1 - t_1| + |y_2 - t_2|] : \text{mean absolute error}$$

$$C = -\frac{1}{2}[t_1 \log(y_1) + (1 - t_1) \log(1 - y_1) + t_2 \log(y_2) + (1 - t_2) \log(1 - y_2)] : \text{cross-entropy (log loss)}$$

$$C = \frac{1}{2}[(y_1 - t_1)^2 + (y_2 - t_2)^2] \quad \longrightarrow \quad \frac{dC}{dy_1} = y_1 - t_1, \quad \frac{dC}{dy_2} = y_2 - t_2$$

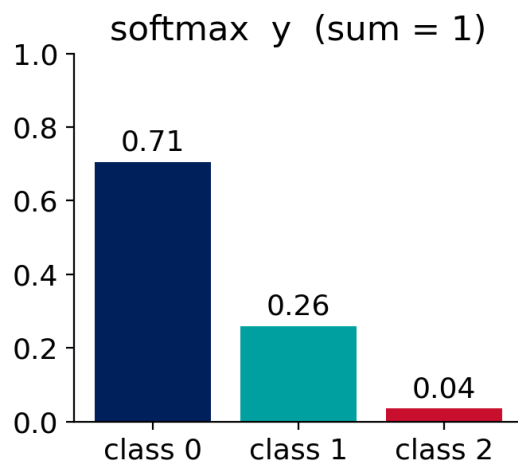
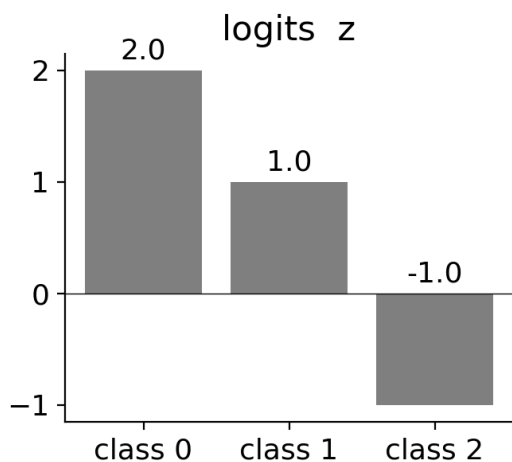
$$\phi(x) = \frac{1}{1 + e^{-x}} \quad \longrightarrow \quad \frac{d\phi(x)}{dx} = \phi(x)(1 - \phi(x))$$

Cost function: softmax

다중 분류: 출력층에 softmax, 비용함수는 cross-entropy

$$y_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$$C = -\sum_k t_k \log y_k$$



예) 정답이 class 0

$t = [1, 0, 0]$

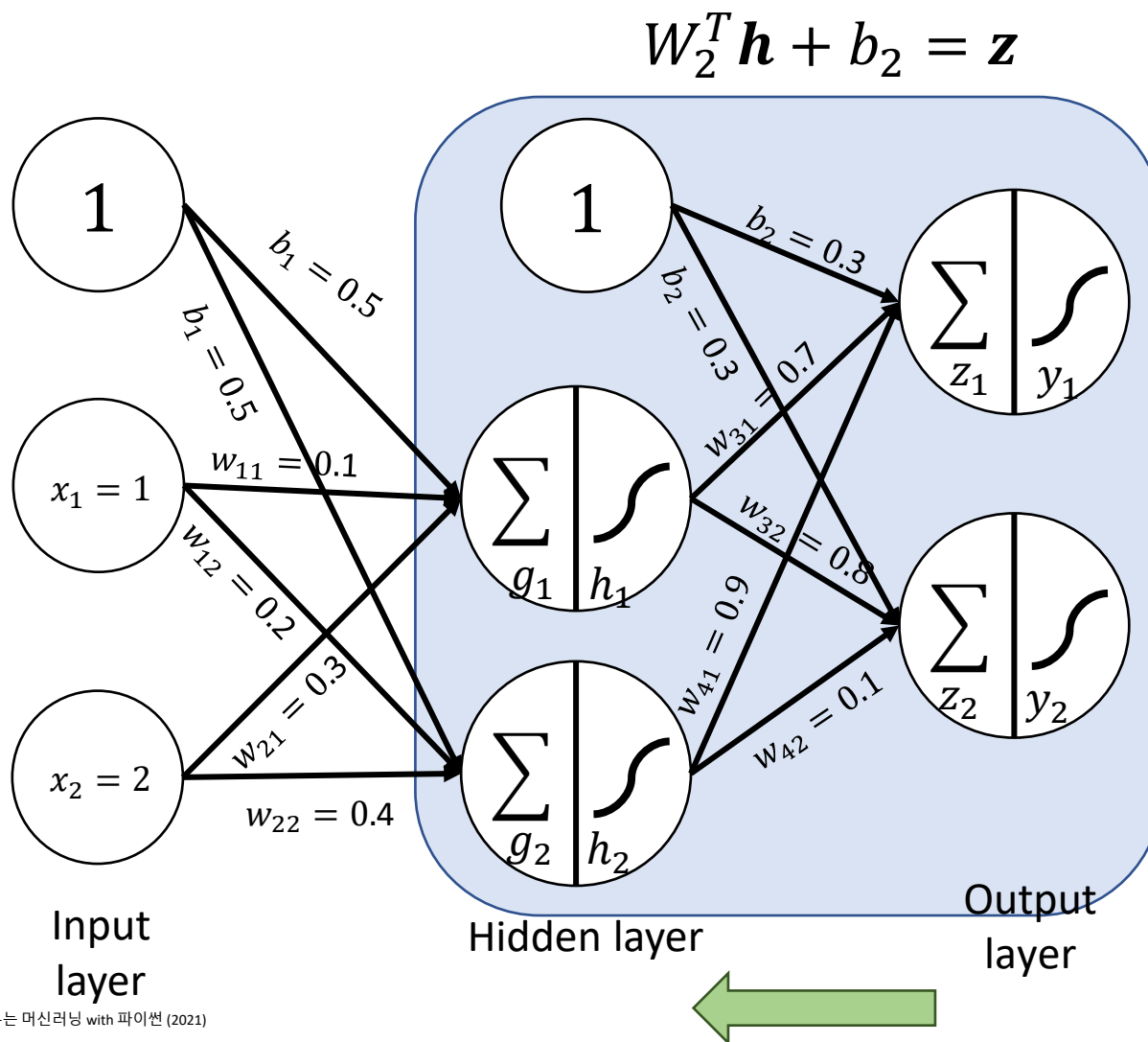
$y = [0.71, 0.26, 0.04]$

$C = -\log 0.71 = 0.35$

- softmax 출력은 합이 1 → 각 class에 속할 확률로 해석
- softmax + cross-entropy 조합의 기울기는 $\partial C / \partial z_k = y_k - t_k$ 로 단순
- 와인 예제 코드: `Activation('softmax'), loss='categorical_crossentropy'`

Learning process

4. Derivative of cost function : back propagation



Learning process

4. Derivative of cost function : back propagation

$$W_2^T \mathbf{h} + b_2 = \mathbf{z}$$

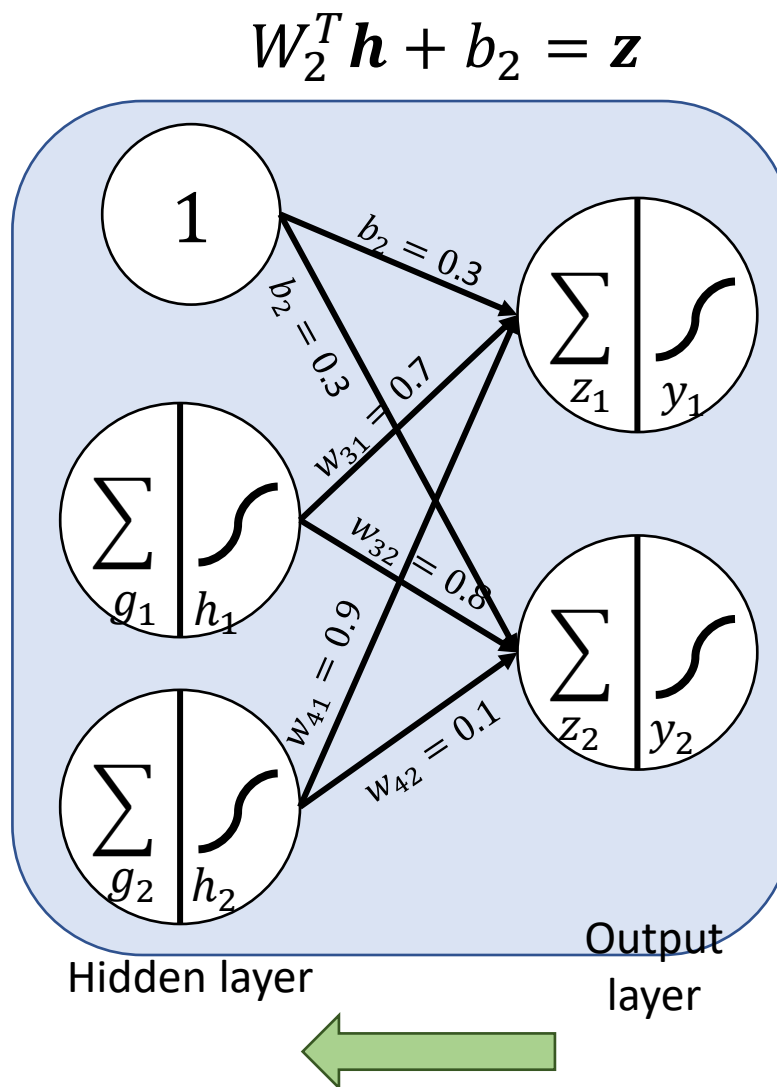
$$\frac{dC}{dw_{31}} = \frac{dC}{dy_1} \frac{dy_1}{dz_1} \frac{dz_1}{dw_{31}}$$

$$\frac{dC}{dw_{32}} = \frac{dC}{dy_2} \frac{dy_2}{dz_2} \frac{dz_2}{dw_{32}}$$

$$\frac{dC}{dw_{41}} = \frac{dC}{dy_1} \frac{dy_1}{dz_1} \frac{dz_1}{dw_{41}}$$

$$\frac{dC}{dw_{42}} = \frac{dC}{dy_2} \frac{dy_2}{dz_2} \frac{dz_2}{dw_{42}}$$

$$\left\{ \begin{array}{l} \frac{dC}{dy_1} = y_1 - t_1, \quad \frac{dC}{dy_2} = y_2 - t_2 : \text{MSE} \\ \frac{dy_1}{dz_1} = y_1(1 - y_1) : \text{sigmoid} \\ \frac{dz}{dW} = \mathbf{h} \end{array} \right.$$



Learning process

4. Derivative of cost function : back propagation

$$W_2^T \mathbf{h} + b_2 = \mathbf{z}$$

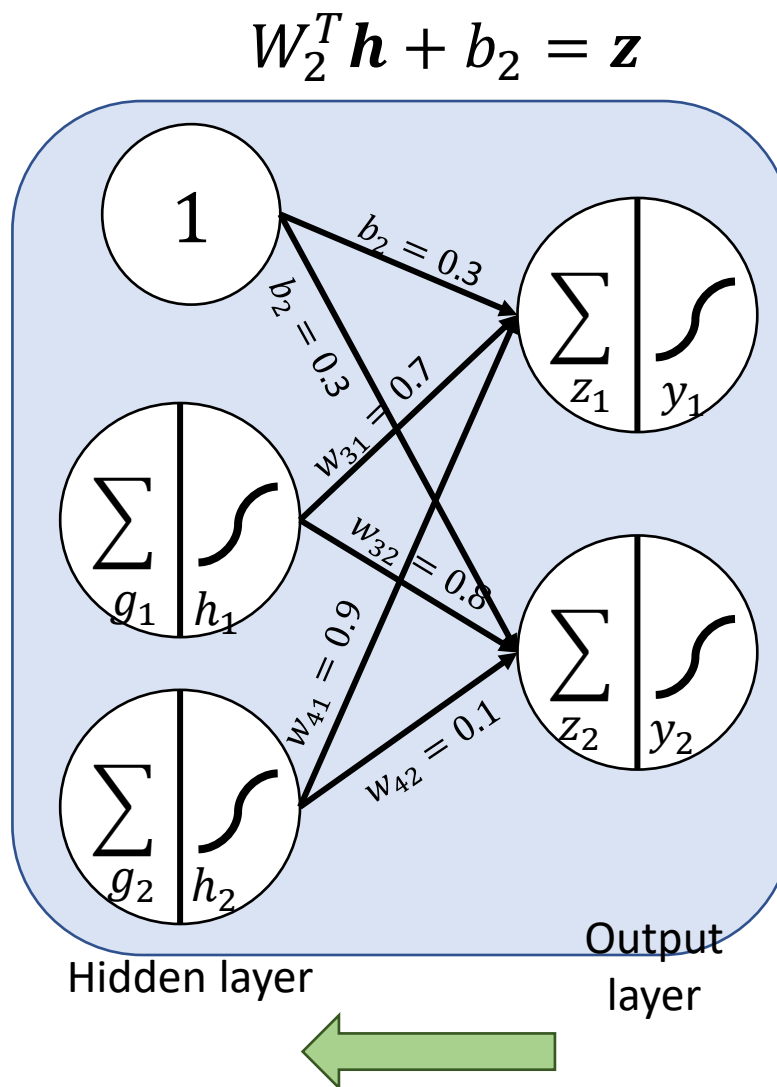
$$\frac{dC}{dw_{31}} = \frac{dC}{dy_1} \frac{dy_1}{dz_1} \frac{dz_1}{dw_{31}}$$

$$\frac{dC}{dw_{32}} = \frac{dC}{dy_2} \frac{dy_2}{dz_2} \frac{dz_2}{dw_{32}}$$

$$\frac{dC}{dw_{41}} = \frac{dC}{dy_1} \frac{dy_1}{dz_1} \frac{dz_1}{dw_{41}}$$

$$\frac{dC}{dw_{42}} = \frac{dC}{dy_2} \frac{dy_2}{dz_2} \frac{dz_2}{dw_{42}}$$

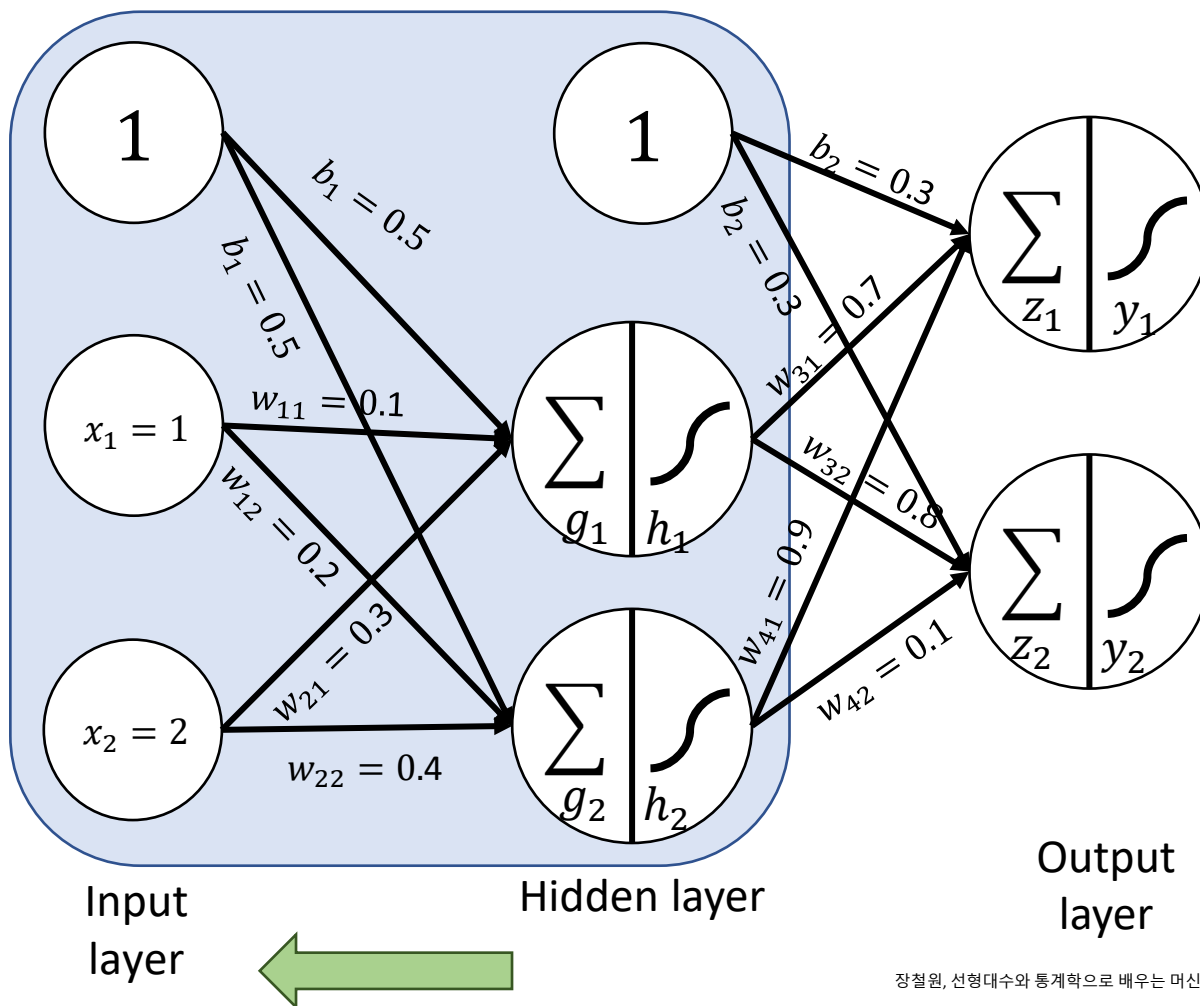
$$\left\{ \begin{array}{l} \frac{dC}{dy_1} = y_1 - t_1, \quad \frac{dC}{dy_2} = y_2 - t_2 : \text{MSE} \\ \frac{dy_1}{dz_1} = y_1(1 - y_1) : \text{sigmoid} \\ \frac{dz}{dW} = \mathbf{h} \end{array} \right.$$



Learning process

4. Derivative of cost function : back propagation

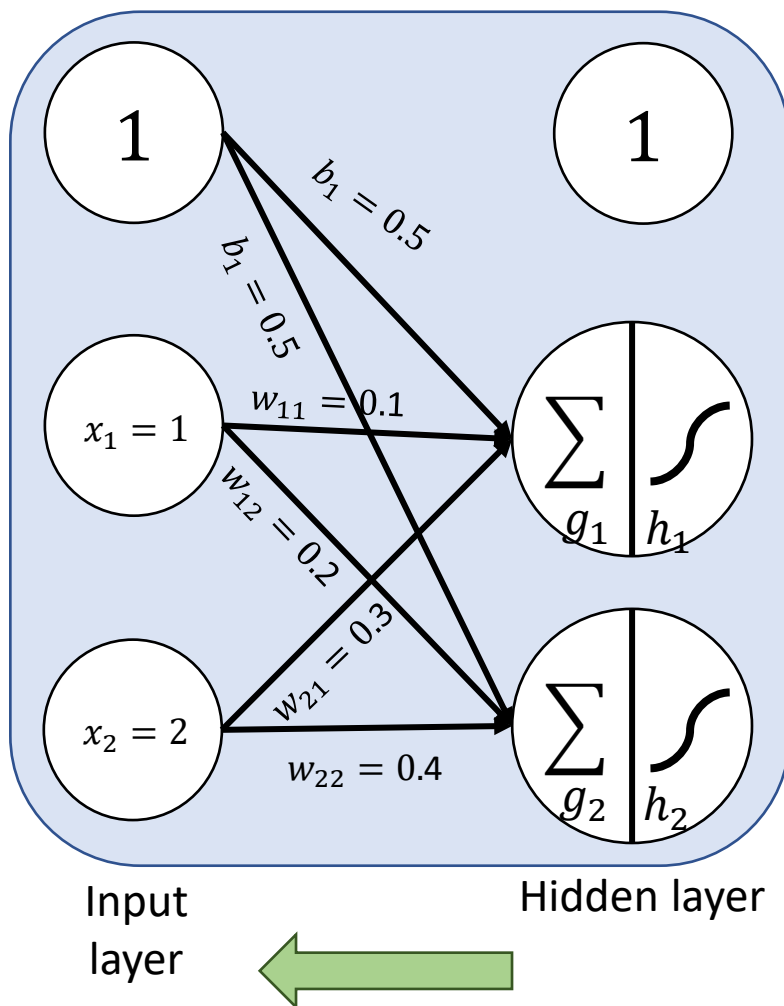
$$W_1^T \mathbf{x} + b_1 = \mathbf{g}$$



Learning process

4. Derivative of cost function : back propagation

$$W_1^T \mathbf{x} + b_1 = \mathbf{g}$$



$$W_1^T \mathbf{x} + b_1 = \mathbf{g}$$

$$\frac{dC}{dw_{11}} = \frac{dC}{dh_1} \frac{dh_1}{dg_1} \frac{dg_1}{dw_{11}}$$

$$\frac{dC}{dw_{21}} = \frac{dC}{dh_1} \frac{dh_1}{dg_1} \frac{dg_1}{dw_{21}}$$

$$\frac{dC}{dw_{12}} = \frac{dC}{dh_2} \frac{dh_2}{dg_2} \frac{dg_2}{dw_{12}}$$

$$\frac{dC}{dw_{22}} = \frac{dC}{dh_2} \frac{dh_2}{dg_2} \frac{dg_2}{dw_{22}}$$

$$\frac{dC}{dh_1} = \frac{dC}{dy_1} \frac{dy_1}{dz_1} \frac{dz_1}{dh_1} + \frac{dC}{dy_2} \frac{dy_2}{dz_2} \frac{dz_2}{dh_1}$$

$$\left\{ \begin{array}{l} \frac{dC}{dy_1} = y_1 - t_1, \quad \frac{dC}{dy_2} = y_2 - t_2 : \text{MSE} \\ \frac{dh_1}{dg_1} = h_1(1 - h_1) : \text{sigmoid} \\ \frac{dg}{d\mathbf{W}} = \mathbf{x} \quad \frac{dz}{d\mathbf{W}} = \mathbf{h} \end{array} \right.$$

Back propagation 계산 예 (1)

정답 $\mathbf{t} = (1, 0)$ 가정, $C = \frac{1}{2}\|\mathbf{y} - \mathbf{t}\|^2 = 0.281$

Output layer: $\delta = (\mathbf{y} - \mathbf{t}) \odot \mathbf{y}(1 - \mathbf{y})$, $\partial C / \partial \mathbf{W}_2 = \mathbf{h} \delta^T$

$$\delta = \begin{matrix} \mathbf{y} - \mathbf{t} \\ \begin{bmatrix} -0.172 \\ 0.730 \end{bmatrix} \end{matrix} \odot \begin{matrix} \mathbf{y}(1 - \mathbf{y}) \\ \begin{bmatrix} 0.142 \\ 0.197 \end{bmatrix} \end{matrix} = \begin{matrix} \delta \\ \begin{bmatrix} -0.0244 \\ 0.1438 \end{bmatrix} \end{matrix}$$

$\odot$: 같은 위치의 원소끼리 곱
 δ_2 가 큰 이유: y_2 가 정답 0에서 멀다

$$\partial C / \partial \mathbf{W}_2 = \begin{matrix} \mathbf{h} \\ \begin{bmatrix} 0.769 \\ 0.818 \end{bmatrix} \end{matrix} \begin{matrix} \delta^T \\ \begin{bmatrix} -0.0244 & 0.1438 \end{bmatrix} \end{matrix} = \begin{bmatrix} -0.0188 & 0.1105 \\ -0.0200 & 0.1176 \end{bmatrix}$$

1행 = $(\partial C / \partial w_{31}, \partial C / \partial w_{32})$

2행 = $(\partial C / \partial w_{41}, \partial C / \partial w_{42})$

- $\partial C / \partial w_{31} < 0 \rightarrow w_{31}$ 을 키우면 cost 감소 (y_1 을 정답 1 쪽으로 올림)
- $\partial C / \partial w_{32} > 0 \rightarrow w_{32}$ 를 줄여야 함 (y_2 를 정답 0 쪽으로 내림)
- 출력 노드마다 δ 를 한 번만 구하면, 그 노드로 들어오는 모든 가중치의 기울기는 $\delta \times$ 입력

Back propagation 계산 예 (2)

Hidden layer: $\partial C/\partial \mathbf{h} = \mathbf{W}_2 \delta$, $\delta_h = \partial C/\partial \mathbf{h} \odot \mathbf{h}(1 - \mathbf{h})$, $\partial C/\partial \mathbf{W}_1 = \mathbf{x} \delta_h^T$

$$\frac{\partial C}{\partial \mathbf{h}} = \begin{matrix} & \mathbf{W}_2 & & \\ & & \delta & \\ \begin{bmatrix} \mathbf{0.7} & \mathbf{0.8} \\ \mathbf{0.9} & \mathbf{0.1} \end{bmatrix} & \begin{bmatrix} -0.0244 \\ 0.1438 \end{bmatrix} & = & \begin{bmatrix} \mathbf{0.0980} \\ -0.0076 \end{bmatrix} \end{matrix}$$

순전파는 $\mathbf{W}_2^T$, 역전파는 $\mathbf{W}_2$
1행: $0.7 \times (-0.0244) + 0.8 \times 0.1438$

$$\delta_h = \begin{matrix} & \partial C/\partial \mathbf{h} & & \mathbf{h}(1 - \mathbf{h}) & & \delta_h \\ \begin{bmatrix} 0.0980 \\ -0.0076 \end{bmatrix} & \odot & \begin{bmatrix} 0.178 \\ 0.149 \end{bmatrix} & = & \begin{bmatrix} 0.0174 \\ -0.0011 \end{bmatrix} \end{matrix}$$

$\mathbf{h}(1 - \mathbf{h})$: sigmoid의 미분

$$\frac{\partial C}{\partial \mathbf{W}_1} = \begin{matrix} & \mathbf{x} & & \delta_h^T & \\ \begin{bmatrix} 1 \\ 2 \end{bmatrix} & \begin{bmatrix} 0.0174 & -0.0011 \end{bmatrix} & = & \begin{bmatrix} \mathbf{0.0174} & \mathbf{-0.0011} \\ 0.0349 & -0.0023 \end{bmatrix} \end{matrix}$$

1행 = $(\partial C/\partial w_{11}, \partial C/\partial w_{12})$

2행 = $(\partial C/\partial w_{21}, \partial C/\partial w_{22})$

- Update ($\eta = 0.5$): w_{31} 0.7 $\rightarrow$ 0.709, w_{32} 0.8 $\rightarrow$ 0.745, w_{11} 0.1 $\rightarrow$ 0.091
- 모든 가중치를 한 번 갱신하면 c: 0.281 $\rightarrow$ 0.256
- 출력층의 δ 를 $\mathbf{W}_2$ 를 거꾸로 타고 은닉층으로 전달 $\rightarrow$ "오차가 뒤로 전파"

Learning process

5. Update parameters

$$w_{11} \rightarrow w_{11} - \eta \frac{dC}{dw_{11}}$$

$$w_{12} \rightarrow w_{12} - \eta \frac{dC}{dw_{12}}$$

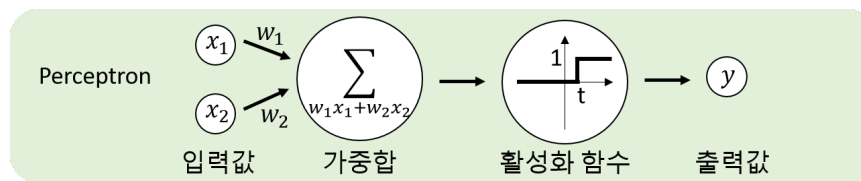
$$w_{21} \rightarrow w_{21} - \eta \frac{dC}{dw_{21}}$$

$$w_{22} \rightarrow w_{22} - \eta \frac{dC}{dw_{22}}$$

....

6. Repeat (1)-(5)

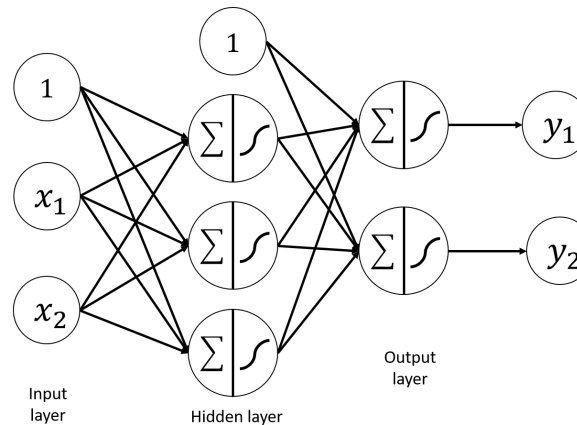
Perceptron: a single neuron, building block of neural networks



Artificial neural network

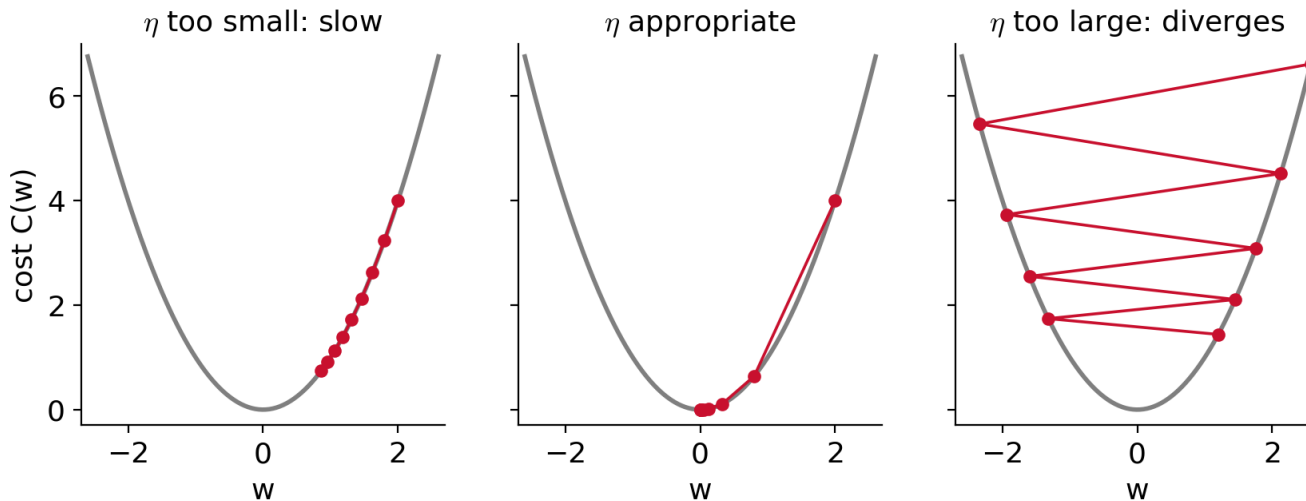
: A collection of artificial neuron, perceptron

: PyTorch, TensorFlow, and MXNet



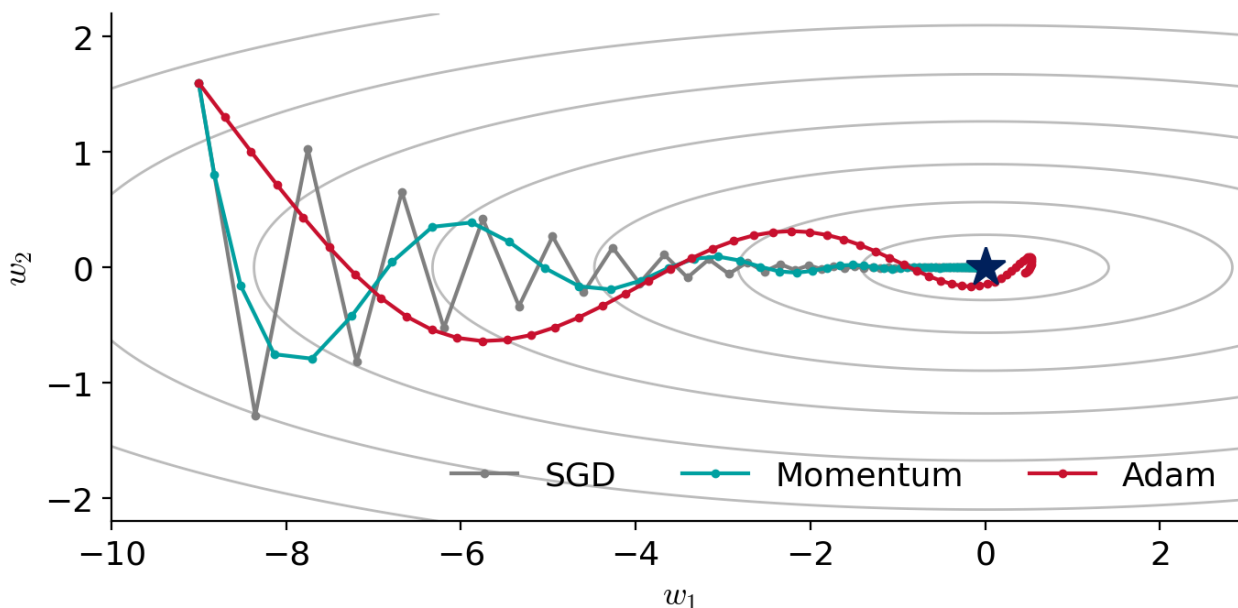
Gradient descent

$$w \leftarrow w - \eta \frac{\partial C}{\partial w}$$



- **Learning rate η :** 한 번에 이동하는 크기. 가장 먼저 조정하는 hyperparameter
- **Batch / stochastic / mini-batch:** 기울기 계산에 쓰는 데이터 수 (전체 / 1개 / 일부)
- **Learning rate schedule:** 처음엔 크게, 뒤로 갈수록 작게 (step decay, cosine, warm-up)

Optimizer



SGD

$$w \leftarrow w - \eta g$$

기울기 방향으로만 이동
좁은 골짜기에서 진동

Momentum

$$v \leftarrow \beta v - \eta g, \quad w \leftarrow w + v$$

이전 이동 방향(관성)을 유지
진동 감소, 수렴 가속

Adam

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}}$$

파라미터별로 step 크기 자동 조절
예제 코드의 optimizer='adam'

Example of ANN (2)

Classification of wine type [0,1,2]

```
#신경망 생성
n_feat = X_tn.shape[1]
n_class = len(set(y))
epo = 30

model = Sequential()
model.add(Dense(20, input_dim=n_feat))
model.add(BatchNormalization())
model.add(Activation('relu'))
model.add(Dense(n_class))
model.add(Activation('softmax'))

#신경망 모형 구조 확인
model.summary()

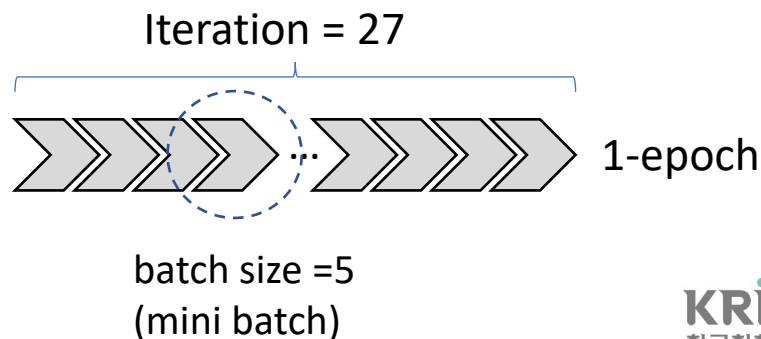
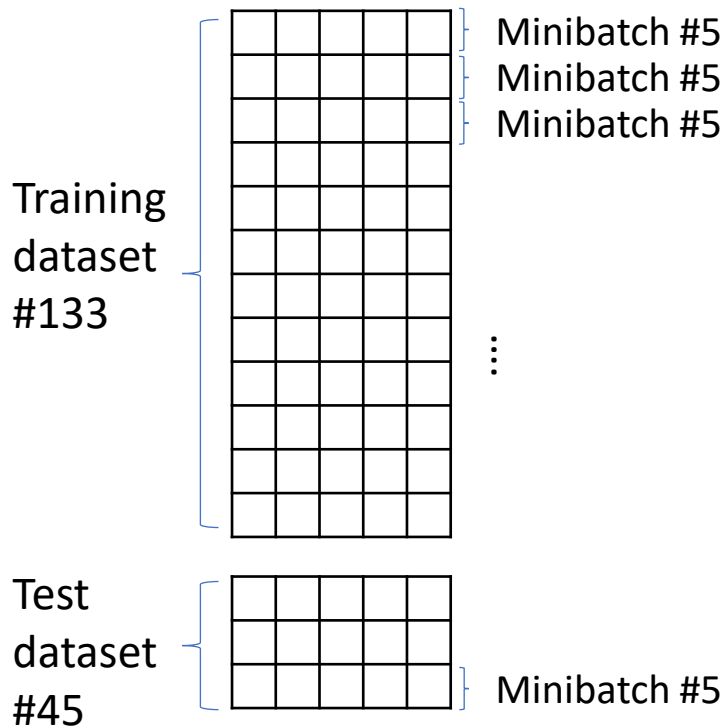
#모형 컴파일
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics='accuracy')

#신경망 학습
hist=model.fit(X_tn, y_tn, epochs=epo, batch_size=5)

#트레이닝 데이터 평가
print(model.evaluate(X_tn, y_tn)[1])

#테스트 데이터 평가
print(model.evaluate(X_te, y_te)[1])
```

장철원, 선형대수와 통계학으로 배우는 머신러닝 with 파이썬 (2021)



Example of ANN (3)

Classification of wine type [0,1,2]

```
#트레이닝 데이터 평가
print(model.evaluate(X_tn, y_tn)[1])

#테스트 데이터 평가
print(model.evaluate(X_te, y_te)[1])

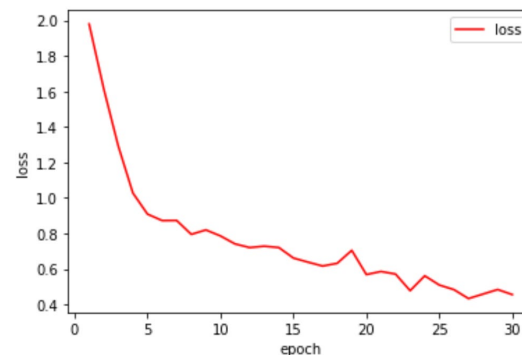
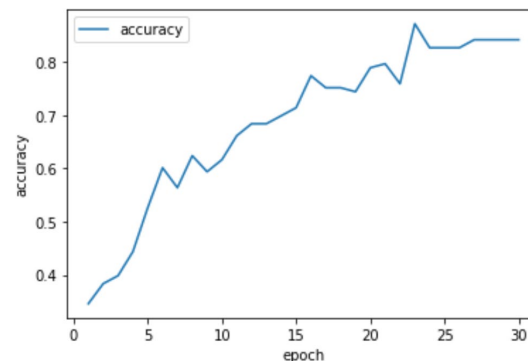
epoch = np.arange(1, epo+1)
accuracy = hist.history['accuracy']
loss = hist.history['loss']

#정확도 학습 그래프
plt.plot(epoch, accuracy, label='accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.show()

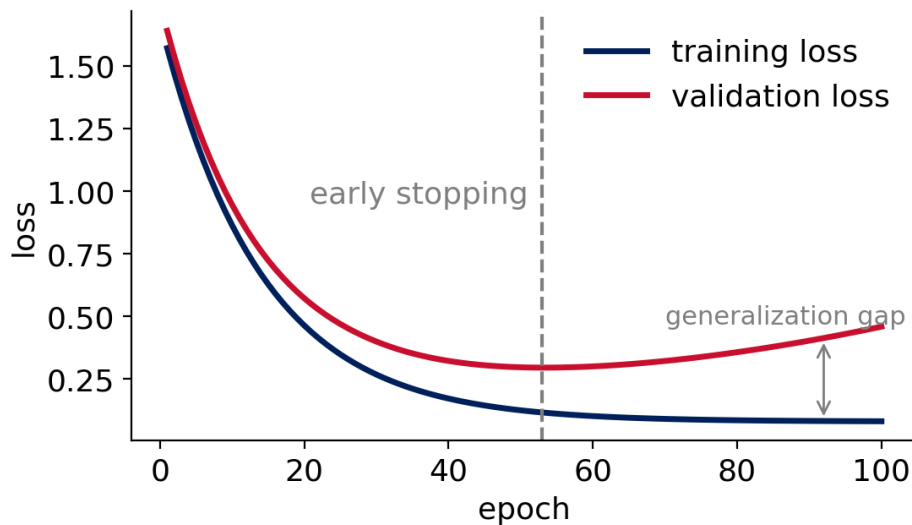
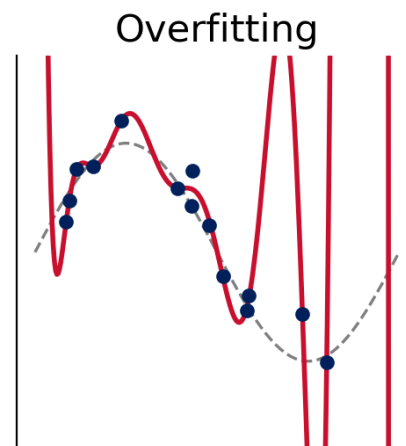
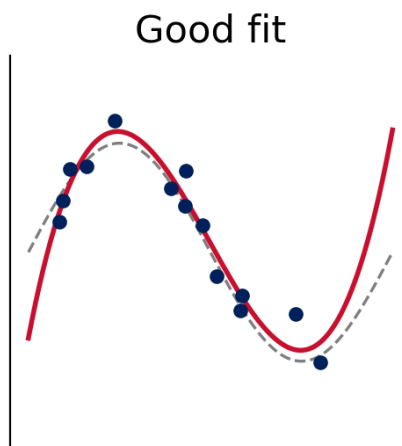
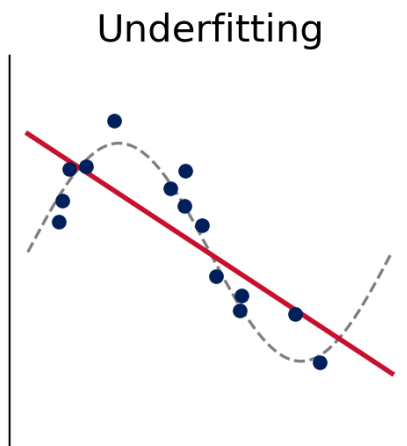
#손실 그래프
plt.plot(epoch, loss, 'r', label='loss')
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend()
plt.show()
```

장철원, 선형대수와 통계학으로 배우는 머신러닝 with 파이썬 (2021)

Training data accuracy: 89.4%
Test data accuracy: 93.3%



Overfitting과 일반화

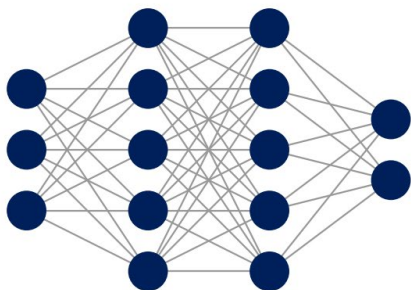


학습 데이터에만 잘 맞는 모델

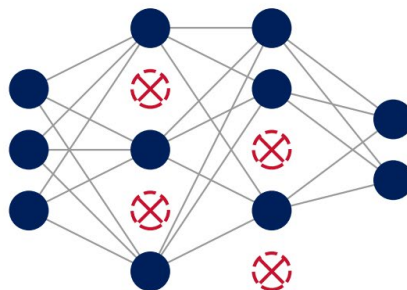
- 데이터를 train / validation / test로 분리
- validation loss가 다시 오르기 시작하면 학습 중단 (early stopping)
- test set은 마지막에 한 번만 사용
- 모델이 클수록, 데이터가 적을수록 과적합 위험 증가

Regularization

Standard network



With dropout (training)



Dropout: 학습할 때마다 뉴런 일부를 확률 p 로 끄
→ 특정 뉴런에 의존하지 않도록

Weight decay (L2)

$$C_{\text{total}} = C + \lambda \sum_i w_i^2$$

큰 가중치에 벌점 → 매끄러운 함수

Batch normalization

층 입력의 평균·분산을 mini-batch 단위로 정규화 → 학습 안정화 (예제 코드의 BatchNormalization)

Early stopping

validation loss 기준으로 학습 중단

Data augmentation / ensemble

이미지: 회전·반전. 소재: 대칭 조작, 여러 모델 평균

소재 데이터의 주의점

Random split: same family in train & test (optimistic error)



Group split: whole families held out (realistic error)



color = chemical family / prototype, red box = test set

- **데이터 누출 (leakage):** 같은 조성계 · 같은 구조 prototype이 train과 test에 함께 들어가면 오차가 낙관적으로 나옴 → group split, 시간순 split
- **데이터 수:** 실험 데이터는 10^2 – 10^3 개 수준인 경우가 많음 → 전통 ML이나 사전학습 모델의 fine-tuning이 유리할 수 있음
- **데이터 일관성:** 단위, 측정 조건, 계산 수준(DFT functional)이 섞이지 않도록 정리. 입력 · 타깃 표준화
- **공정한 비교:** 표준 벤치마크와 정해진 split 사용 (Matbench)

Quick check

Q1

단층 퍼셉트론으로 XOR 문제를 풀 수 없는 이유는?

Q2

은닉층의 활성화 함수를 모두 선형 함수로 바꾸면, 깊은 신경망은 무엇과 같아지는가?

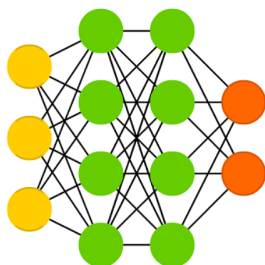
Q3

학습 데이터 133개, batch size 5, epoch 30 → 가중치 update는 모두 몇 번 일어나는가?

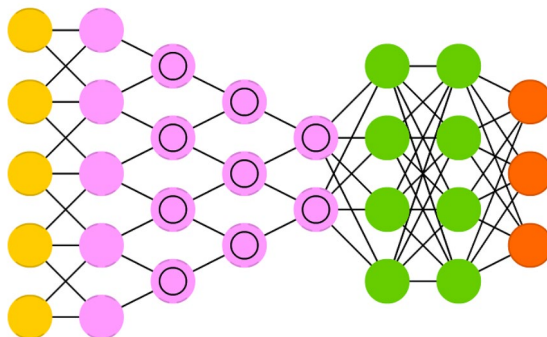
Q4

training accuracy는 계속 오르는데 validation accuracy는 떨어진다. 어떤 조치를 해볼 수 있는가?

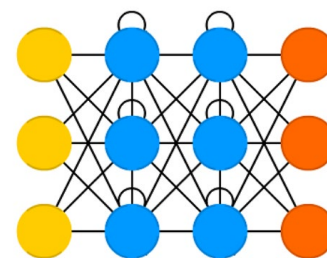
Artificial neural networks



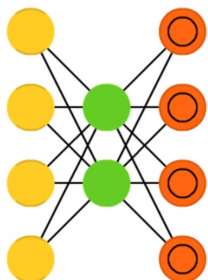
Deep feedforward network (DFN)



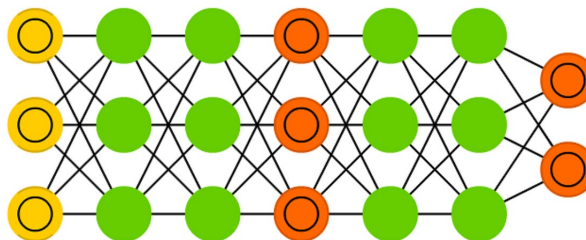
Convolutional neural network (CNN)



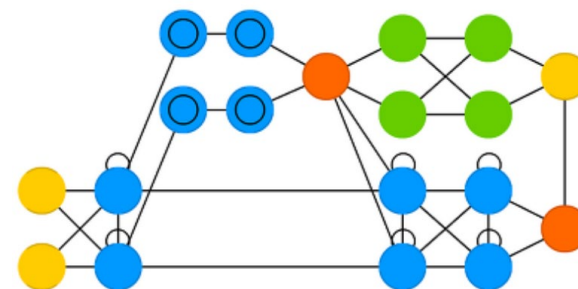
Recurrent neural network (RNN)



autoencoder (AE)



Generative adversarial network (GAN)



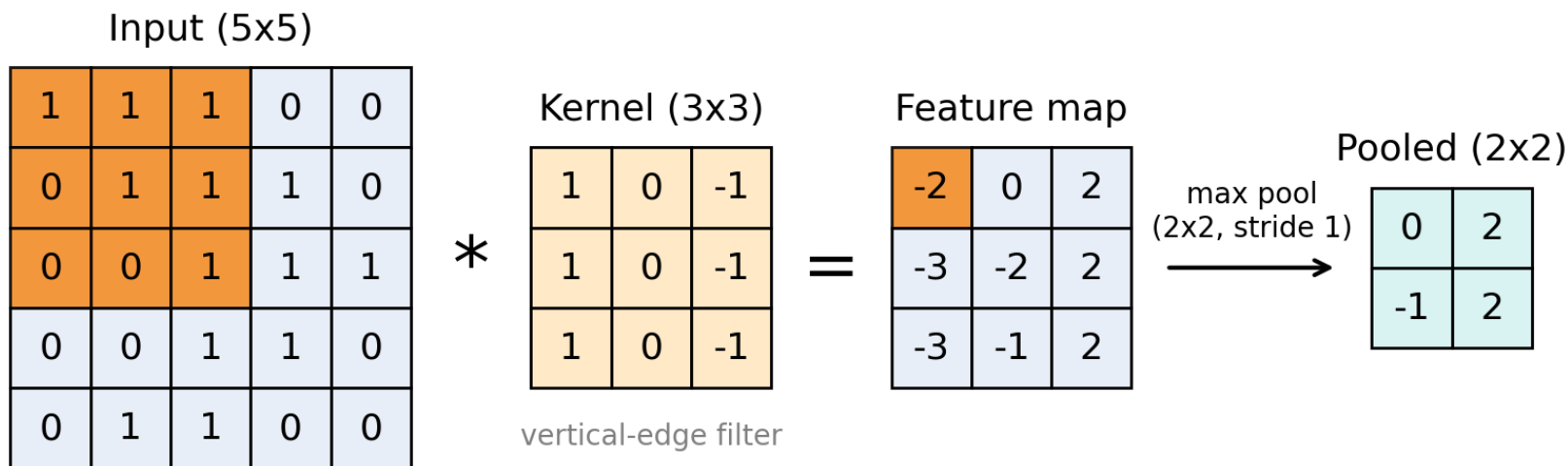
Attention network (AN)

데이터 유형과 신경망 구조

데이터 유형	예	주로 쓰는 구조	소재 연구 예
조성 · 공정 조건 (표)	조성식, 합성 조건	다층 퍼셉트론 (Deep neural network)	ElemNet, Roost
이미지	SEM, TEM/STEM, RHEED	CNN	RHEED 패턴 분류
스펙트럼 · 시계열	XRD, XAS, Raman, in-situ 신호	1D CNN, RNN	XRD 상(phase) 분류
문자열 · 문헌	SMILES, 논문 텍스트	RNN, Transformer	분자 생성, 문헌 마이닝
원자 구조	분자, 결정, 표면	GNN	CGCNN, MEGNet, M3GNet
새 구조 생성	결정 구조, 분자	VAE, GAN, diffusion	CDVAE, MatterGen

데이터의 구조(격자, 순서, 그래프)에 맞는 신경망을 고르는 것이 출발점

CNN: convolution, pooling



- kernel(필터)을 이미지 위에서 이동시키며 가중합 → feature map
- 같은 kernel을 모든 위치에서 공유 → 파라미터 수 감소, 패턴의 위치가 바뀌어도 인식
- kernel 값은 사람이 정하지 않고 역전파로 학습
- Pooling: 해상도를 줄이면서 주요 특징만 남김
- 다음 슬라이드: RHEED 회절 패턴 이미지 분류

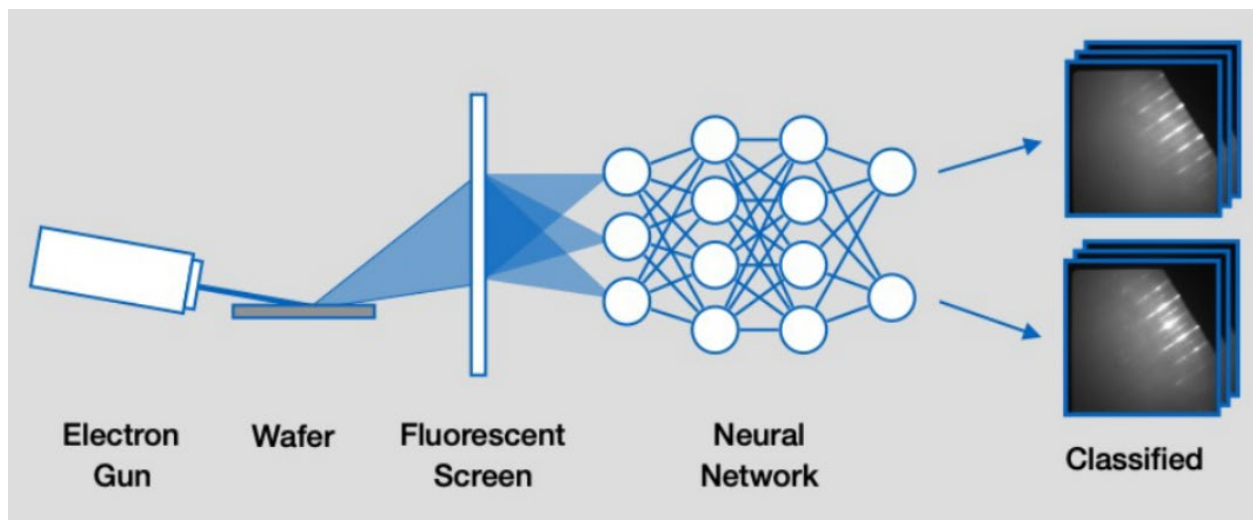
Surface structure pattern of RHEED image

Classification of Reflection High-Energy Electron Diffraction Pattern Using Machine Learning

Jinkwan Kwoen* and Yasuhiko Arakawa

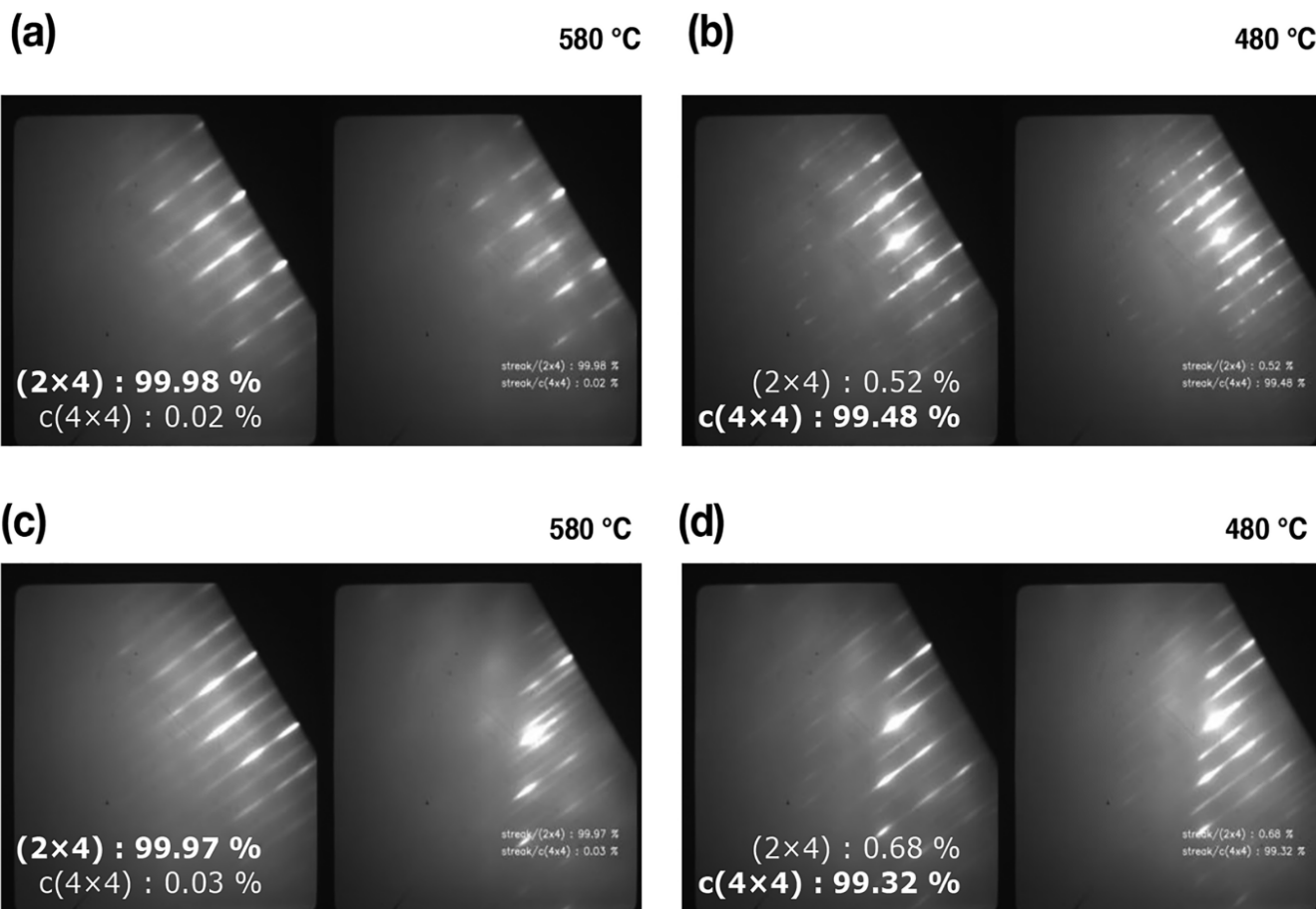
Cryst. Growth Des. 2020, 20, 5289–5293

- GaAs (2x4) and c(4x4) surface structure pattern
- Convolutional neural network (CNN)-based classification
- Source code provided

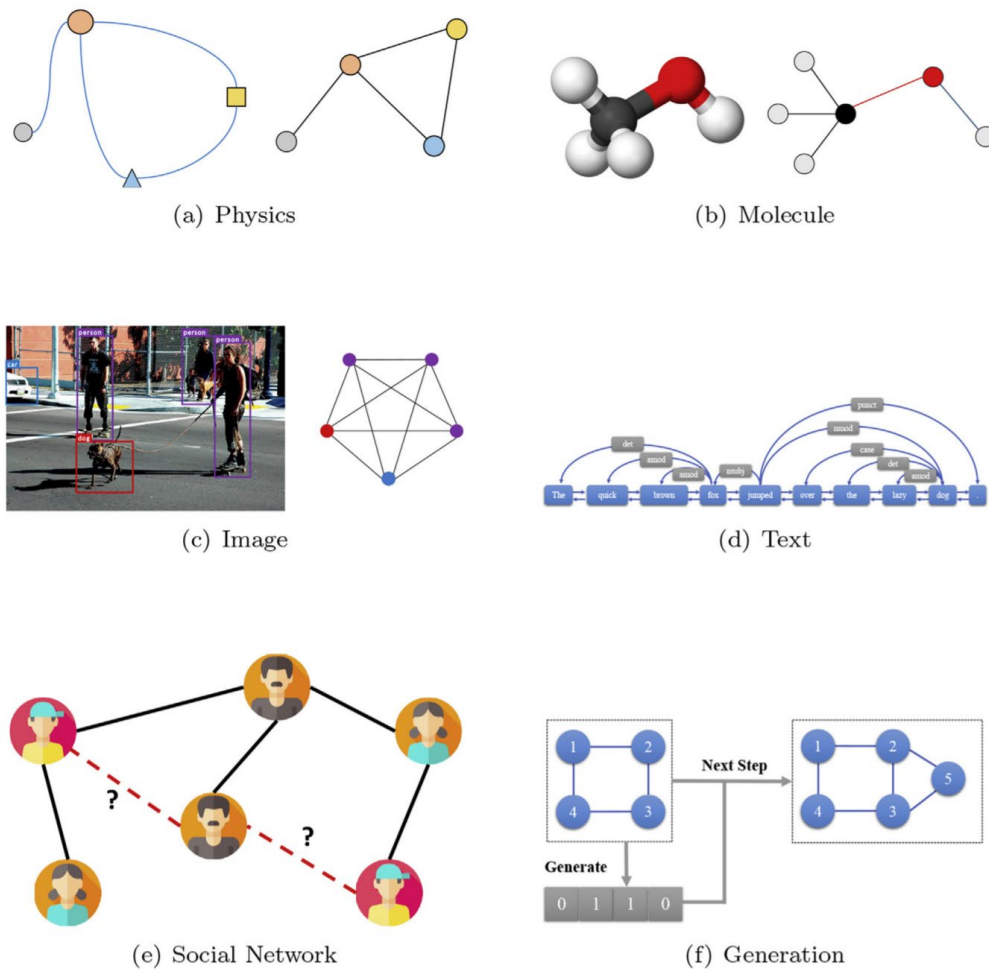


Surface structure pattern of RHEED image

Test results: classification of surface structure pattern of RHEED image



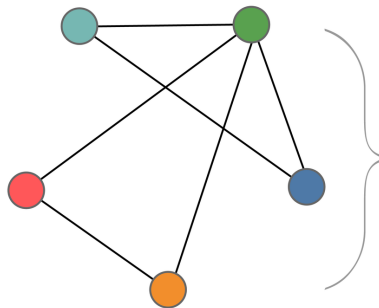
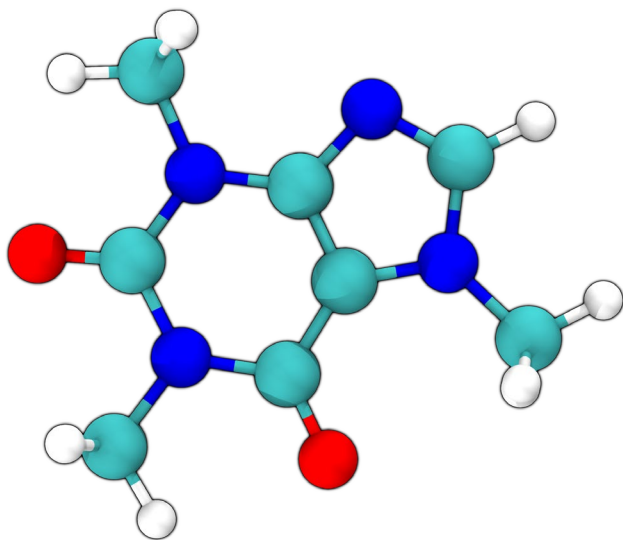
Graph neural network



Graph: node + connectivity

Fig. 6. Application scenarios. (Icons made by Freepik from Flaticon)

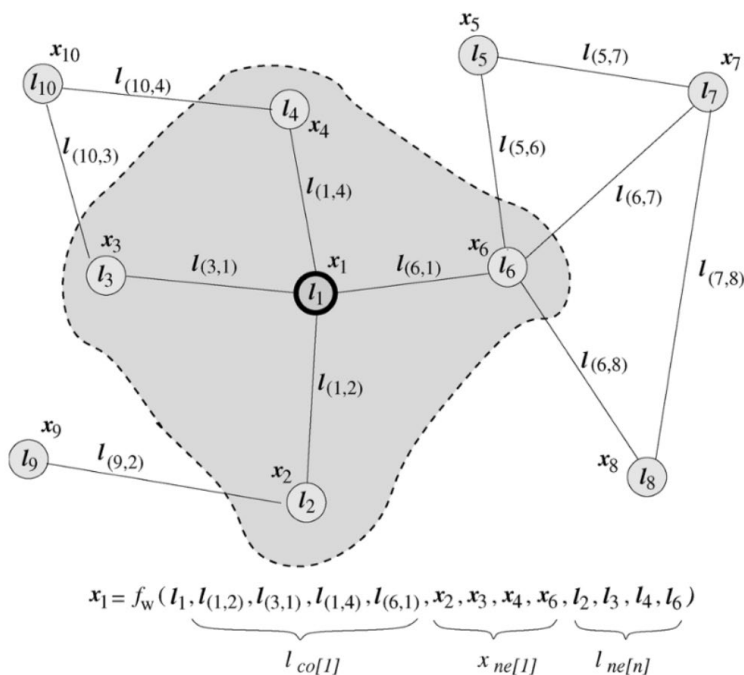
구조의 그래프 표현



- U Global (or master node) attributes
e.g., number of nodes, longest path
- N Vertex (or nodes) attributes
e.g., node identity, number of neighbors
- E Edge (or link) attributes and directionality
e.g., edge identity, edge weight

- **Node** = 원자, **Edge** = 결합 또는 cutoff 반경 안의 이웃, **Global** = 계 전체의 정보
- 원자 번호 매기는 순서를 바꿔도 같은 물질 → permutation invariance
- 평행이동 · 회전해도 물성은 같음 → 좌표 대신 거리 · 각도를 특성으로 사용
- 결정: 주기 경계 조건 때문에 같은 원자 쌍 사이에 edge가 여러 개 생길 수 있음

Graph representation



An illustration of node state update based on the information in its neighbors. Figure from "The Graph Neural Network Model".

State of the n th node

$$\mathbf{x}_n = f_w(\mathbf{l}_n, \mathbf{l}_{co[n]}, \mathbf{x}_{ne[n]}, \mathbf{l}_{ne[n]})$$

$\mathbf{l}_n$ A feature vector of n th node

$\mathbf{l}_{co[n]}$ Feature of connected **edges** with n th node

$\mathbf{x}_{ne[n]}$ The states connected with n th node

$\mathbf{l}_{ne[n]}$ Feature of **neighbor nodes** of n th node

Output, i.e., a decision about the concept

$$\mathbf{o}_n = g_w(\mathbf{x}_n, \mathbf{l}_n)$$

Graph neural network (GNN)

MAIN

```

initialize  $w$ ;
 $x = \text{Forward}(w)$ ;
repeat
     $\frac{\partial \epsilon_w}{\partial w} = \text{BACKWARD}(x, w)$ ;
     $w = w - \lambda \cdot \frac{\partial \epsilon_w}{\partial w}$ ;
     $x = \text{FORWARD}(w)$ ;
until (a stopping criterion);
return  $w$ ; 가중치
    
```

end

FORWARD(w)

```

initialize  $x(0)$ ,  $t = 0$ ;
repeat
     $x(t+1) = F_w(x(t), l)$ ;
     $t = t + 1$ ;
until  $\|x(t) - x(t-1)\| \leq \epsilon_f$ 
return  $x(t)$ ;
    
```

end

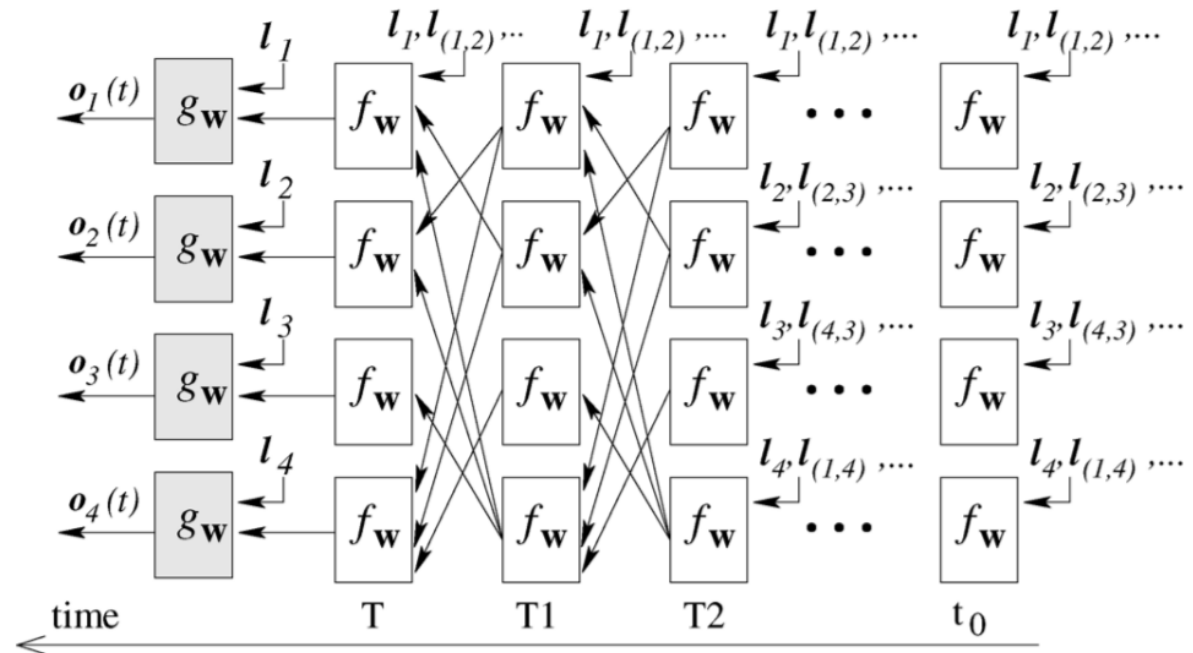
BACKWARD(x, w)

```

 $o = G_w(x, l_N)$ ;
 $A = \frac{\partial F_w}{\partial x}(x, l)$ ;
 $b = \frac{\partial \epsilon_w}{\partial o} \cdot \frac{\partial G_w}{\partial x}(x, l_N)$ ;
initialize  $z(0)$ ,  $t=0$ ;
repeat
     $z(t) = z(t+1) \cdot A + b$ ;
     $t = t - 1$ ;
until  $\|z(t-1) - z(t)\| \leq \epsilon_b$ ;
 $c = \frac{\partial \epsilon_w}{\partial o} \cdot \frac{\partial G_w}{\partial w}(x, l_N)$ ;
 $d = z(t) \cdot \frac{\partial F_w}{\partial w}(x, l)$ ;
 $\frac{\partial \epsilon_w}{\partial w} = c + d$ ;
return  $\frac{\partial \epsilon_w}{\partial w}$ ;
    
```

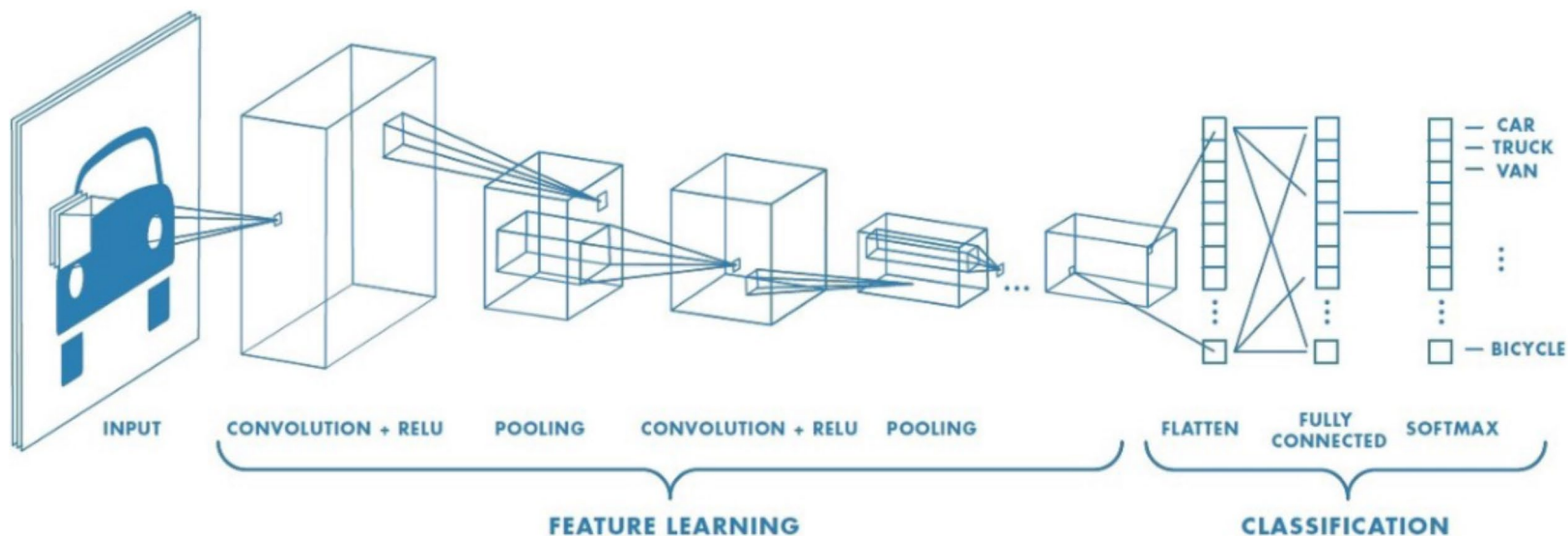
end

$$x_n = f_w(l_n, l_{\text{co}[n]}, x_{\text{ne}[n]}, l_{\text{ne}[n]}) \quad o_n = g_w(x_n, l_n)$$



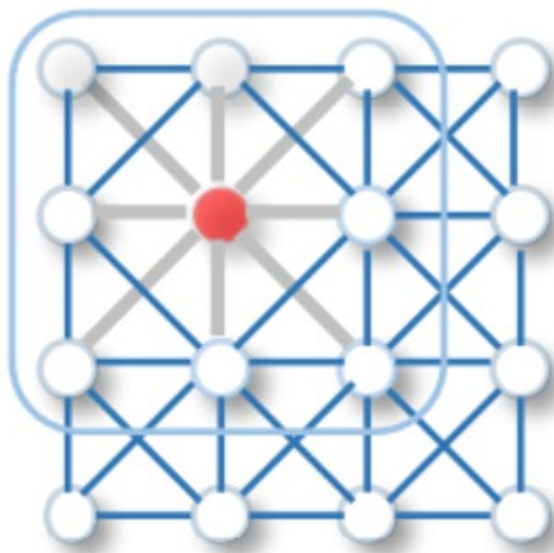
Graph convolutional network (GCN)

Propagation of feature information from neighboring nodes in every layer



<Source : <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>>

Graph convolutional network (GCN)

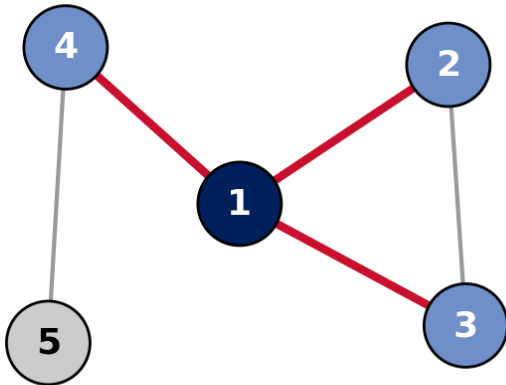


<Source: A Comprehensive Survey on Graph Neural Networks>

GCN layer 수식

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)})$$

node 1 aggregates neighbors 2, 3, 4 (+ itself)



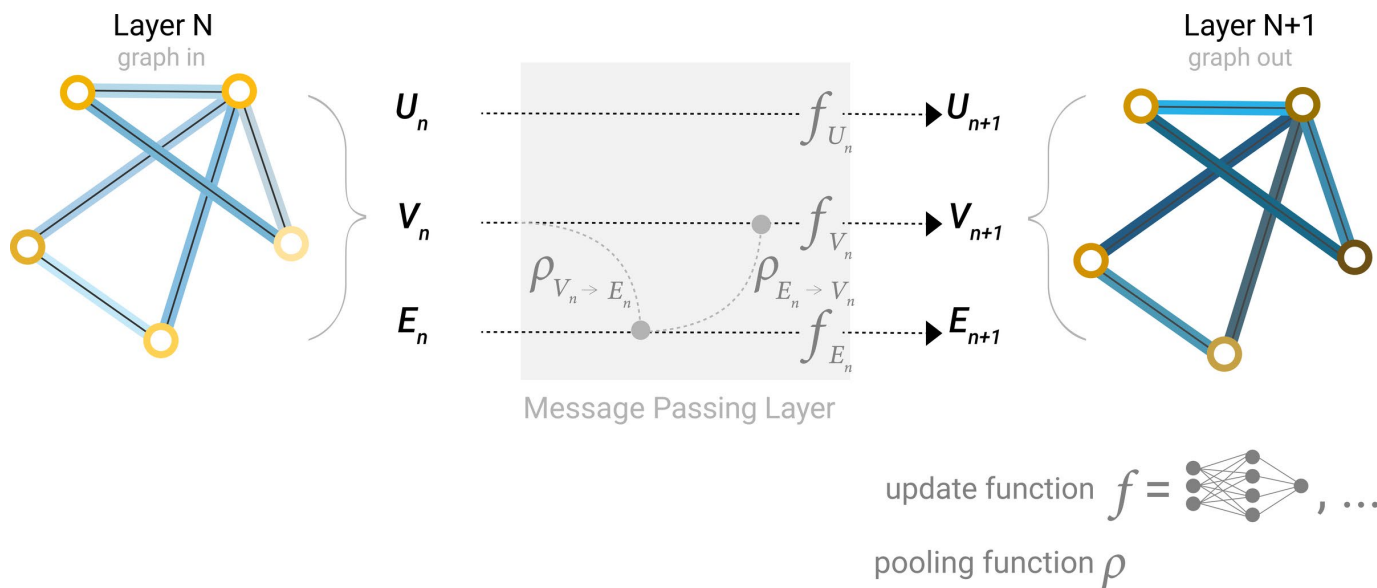
	1	2	3	4	5
1	1	1	1	1	0
2	1	1	1	0	0
3	1	1	1	0	0
4	1	0	0	1	1
5	0	0	0	1	1

$\tilde{A} = A + I$ (adjacency + self-loop)

- H: node 특성 행렬, W: 학습되는 가중치, $\tilde{D}$: $\tilde{A}$ 의 degree 행렬 (이웃 수로 정규화)
- layer 1개 = 바로 옆 이웃(1-hop)의 특성을 평균. L개를 쌓으면 L-hop 떨어진 원자의 정보까지 반영
- 모든 node가 같은 W를 공유 → CNN의 kernel 공유와 같은 발상

Message passing

$$m_i^{t+1} = \sum_{j \in N(i)} M_t(h_i^t, h_j^t, e_{ij}) \quad h_i^{t+1} = U_t(h_i^t, m_i^{t+1}) \quad \hat{y} = R(\{h_i^T\})$$



- ① **Message:** 이웃 node와 edge의 정보를 모음
- ② **Update:** 모은 정보로 자신의 상태를 갱신
- ③ **Readout:** 전체 node를 모아 물성 예측
- GCN, CGCNN, MEGNet, SchNet 등은 M, U, R을 어떻게 선택했는가의 차이

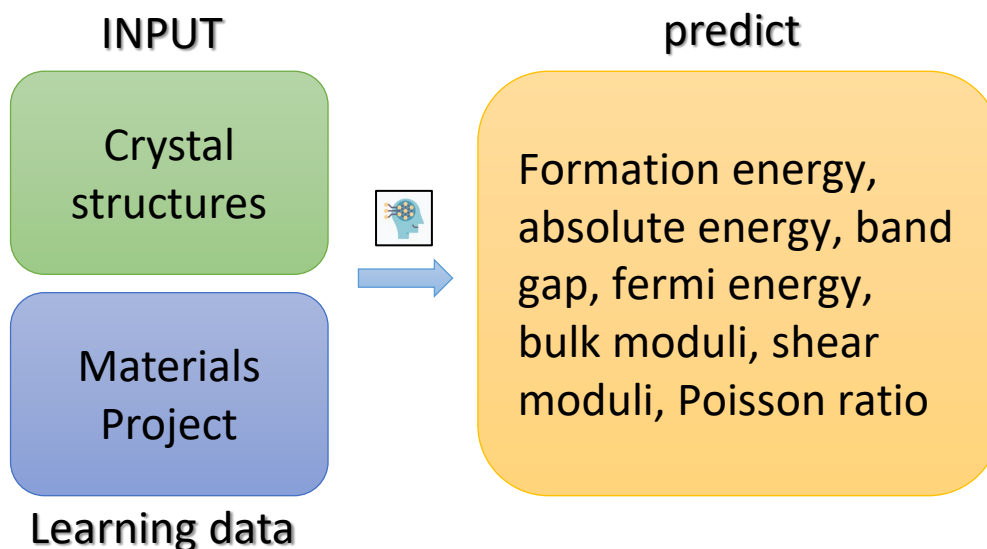
Prediction of material properties with crystal graph

PHYSICAL REVIEW LETTERS **120**, 145301 (2018)

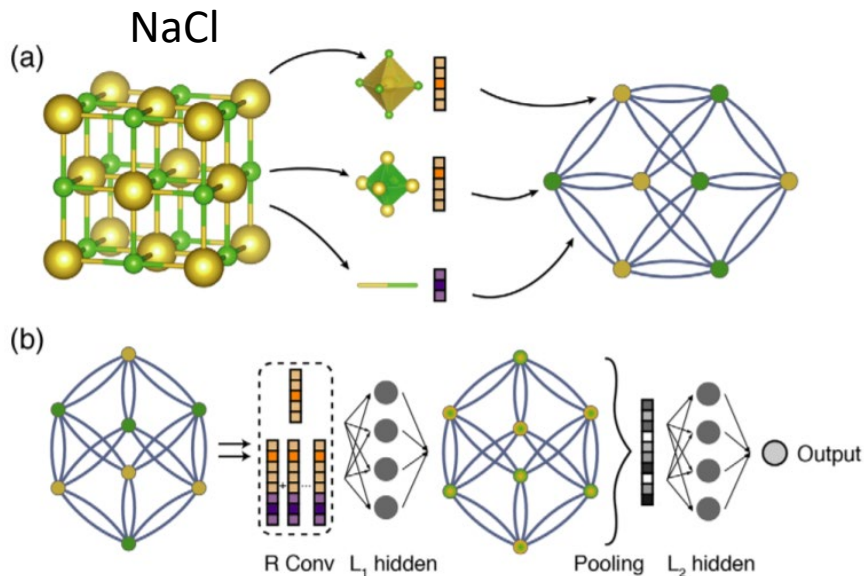
Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties

Tian Xie and Jeffrey C. Grossman

*Department of Materials Science and Engineering, Massachusetts Institute of Technology,
Cambridge, Massachusetts 02139, USA*



Crystal graph convolutional neural network (CGCNN)



Each node i
Feature vector $\mathbf{v}_i$

Each edge $(i, j)_k$
Feature vector $\mathbf{u}_{(i,j)_k}$

Convolution with surrounding atoms and bonds

$$\mathbf{v}_i^{(t+1)} = \text{Conv}\left(\mathbf{v}_i^{(t)}, \mathbf{v}_j^{(t)}, \mathbf{u}_{(i,j)_k}\right), \quad (i, j)_k \in \mathcal{G}.$$

The pooling layer produces an overall feature vector $\mathbf{v}_c$ for the crystal.
(pooling function: a normalized summation)

$$\mathbf{v}_c = \text{Pool}(\mathbf{v}_0^{(0)}, \mathbf{v}_1^{(0)}, \dots, \mathbf{v}_N^{(0)}, \dots, \mathbf{v}_N^{(R)})$$

CGCNN feature generation (atom_init.json)

Elemental properties

Nickel		
Group	10	1.91
Period	4	6.59
		Electronegativity
		Atomic volume

One-hot encoding

Group: 1-18

0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Period: 1-9

0	0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---

EN: 1.2-3.2

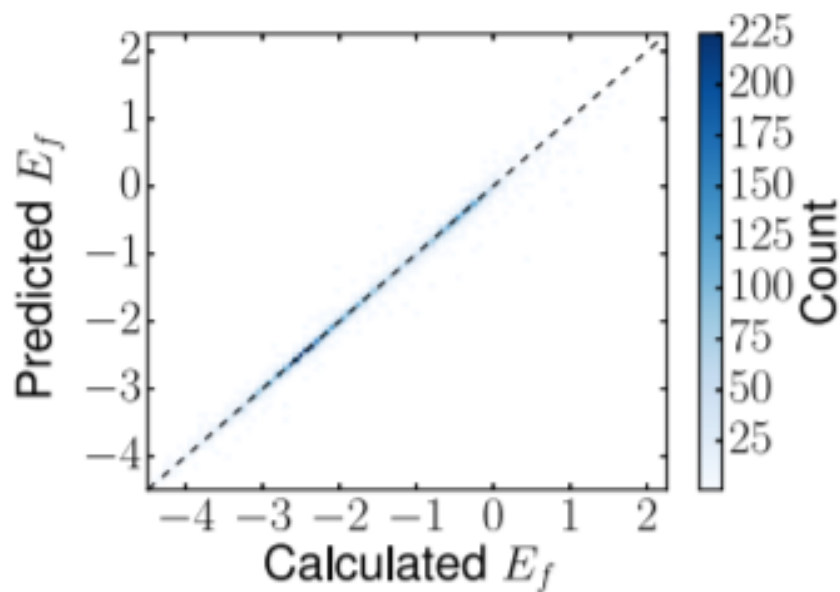
0	0	0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

AV: 6.5-18

1	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

```
{
  "1": [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
    0, 1, 0, 0, 0, 0, 0, 0],
  "2": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
  "3": [0, 1, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
    1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
  "4": [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
    0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
    0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0]
}
```

CGCNN vs DFT calculations

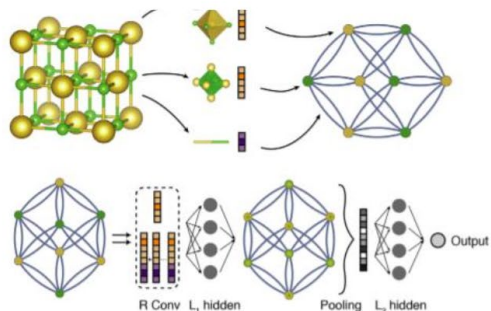


Property	# of train data	Unit	MAE _{model}	MAE _{DFT}
Formation energy	28 046	eV/atom	0.039	0.081–0.136 [28]
Absolute energy	28 046	eV/atom	0.072	...
Band gap	16 458	eV	0.388	0.6 [32]
Fermi energy	28 046	eV	0.363	...
Bulk moduli	2041	log(GPa)	0.054	0.050 [13]
Shear moduli	2041	log(GPa)	0.087	0.069 [13]
Poisson ratio	2041	...	0.030	...

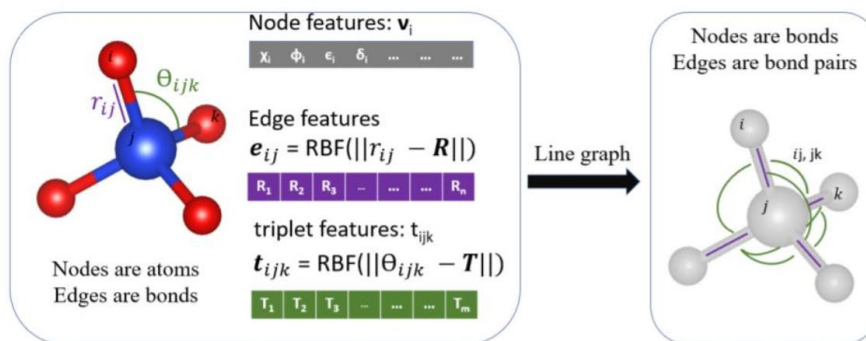
Modification of CGCNN (1)

Atomistic line graph neural network (ALIGNN)

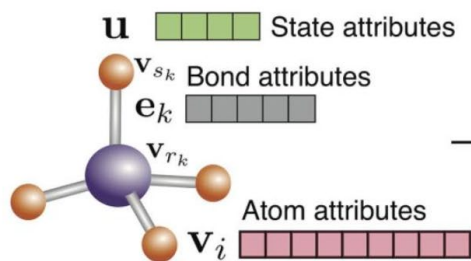
- Formation energy for solids
- Internal energy for molecules



(a) CGCNN

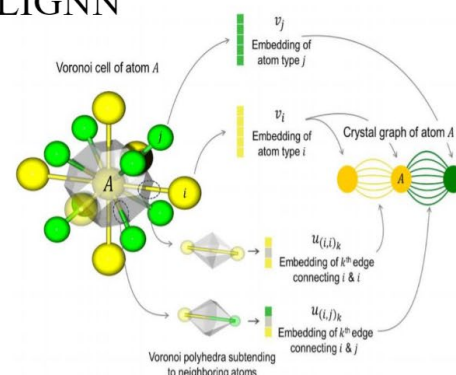


(b) ALIGNN



(c) MEGNet

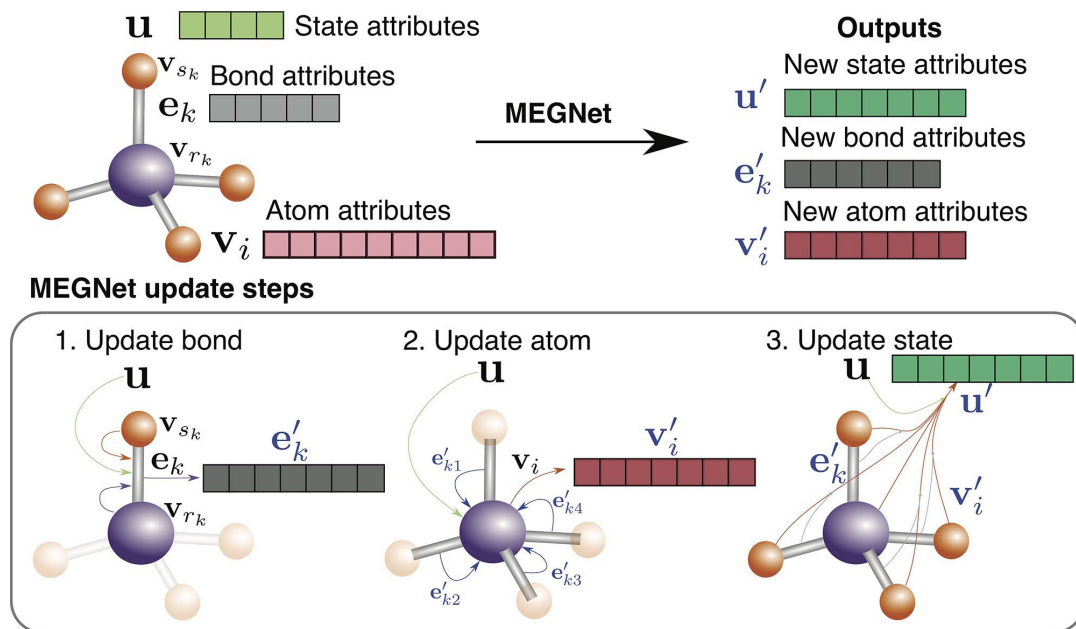
Materials graph neural network
(MEGNet) QM9 & MP datasets



(d) iCGCNN

Improved crystal graph
convolutional neural network

MEGNet

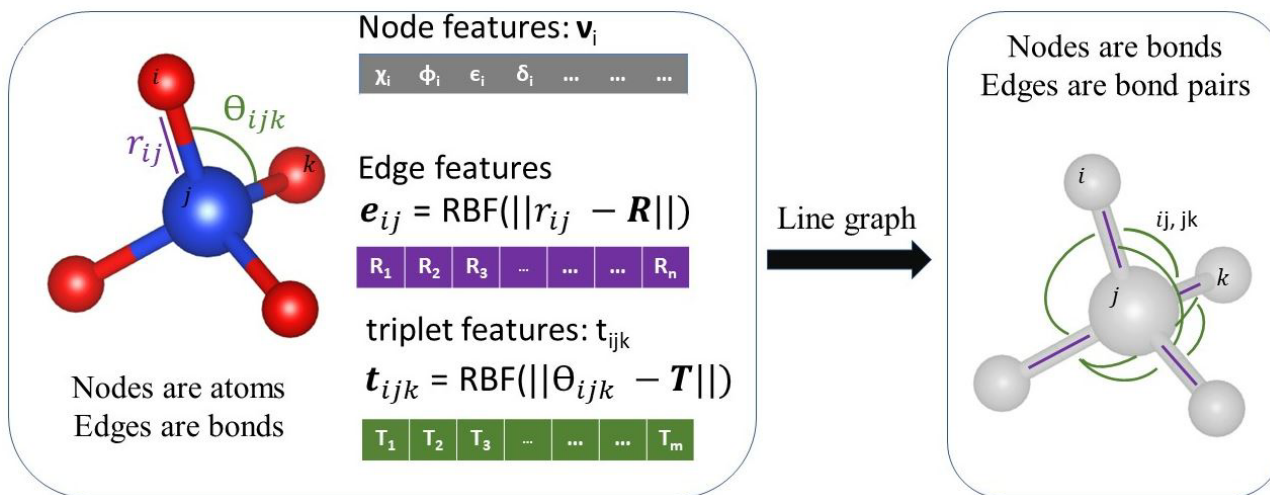


MatERials Graph Network

- atom, bond에 더해 global state $\mathbf{u}$ (온도, 압력 등) 사용
- bond $\rightarrow$ atom $\rightarrow$ state 순서로 update
- 분자(QM9)와 결정(Materials Project)을 하나의 틀로 학습
- 학습된 원소 embedding을 데이터가 적은 물성(band gap, 탄성 계수)에 transfer learning

CGCNN과의 차이: edge(bond) 특성도 매 layer 갱신하고, 계 전체를 나타내는 state를 함께 학습

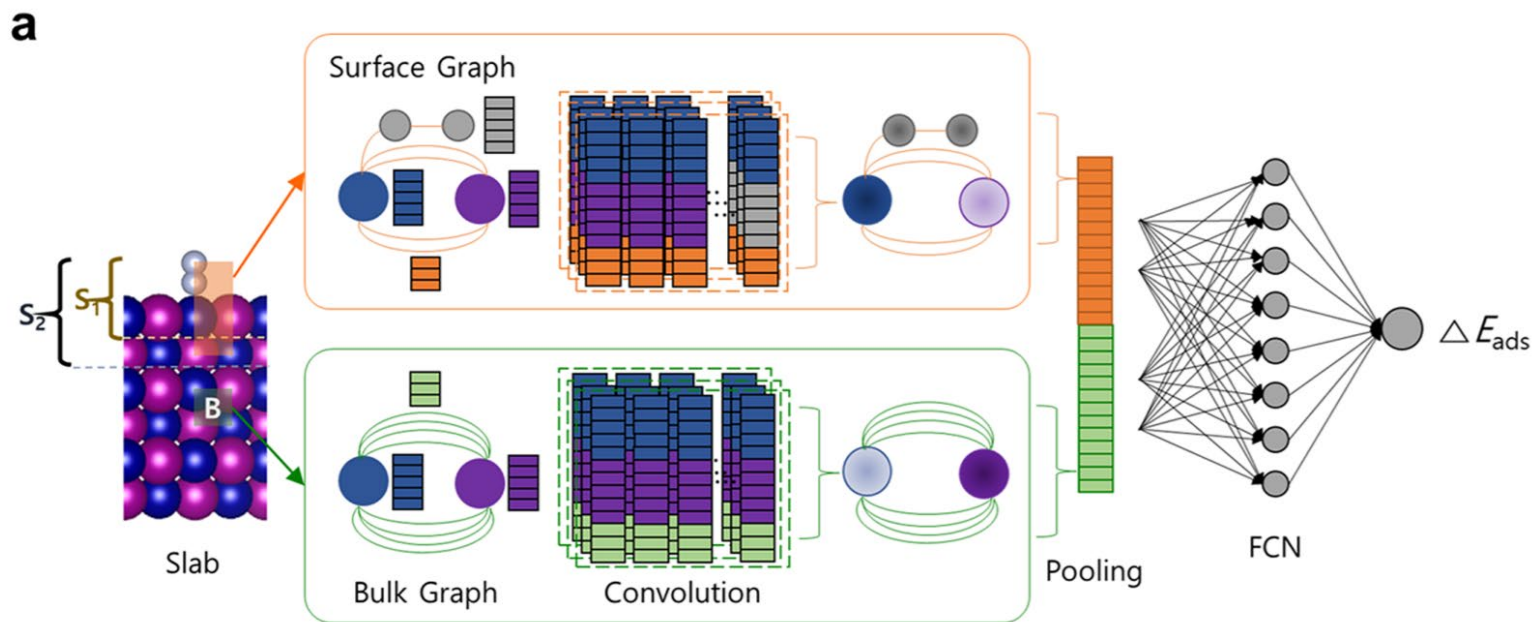
ALIGNN



- 원자 그래프(node = 원자, edge = 결합)와 line graph(node = 결합, edge = 결합쌍의 각도)를 번갈아 update
- 거리만으로는 구분되지 않는 국소 구조(결합각)를 직접 학습
- Materials Project 형성에너지 MAE (논문 보고값, eV/atom): CGCNN 0.039 → MEGNet 0.028 → ALIGNN 0.022

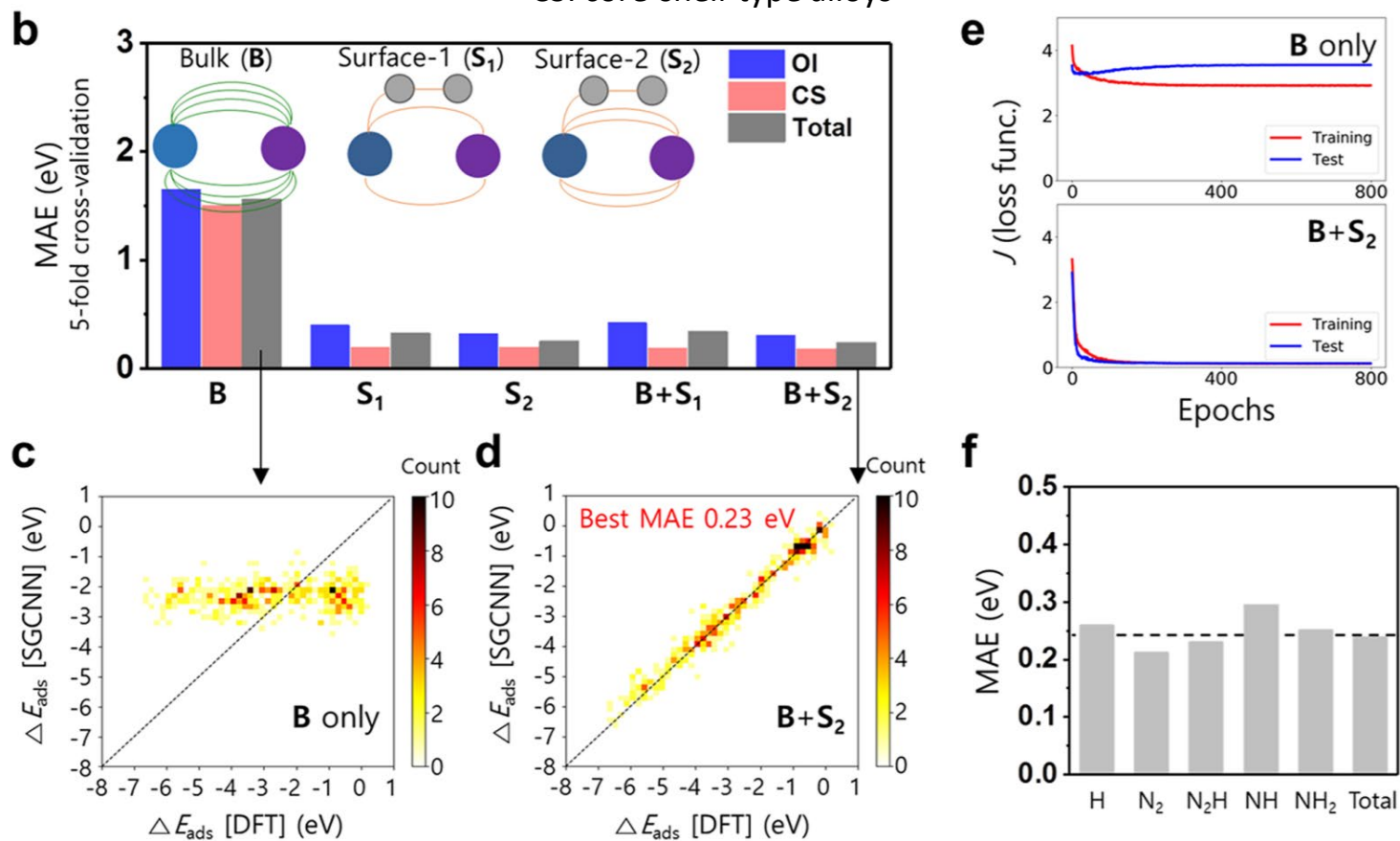
Modification of CGCNN (2)

a slab graph convolutional neural network (SGCNN) for gas adsorption energy

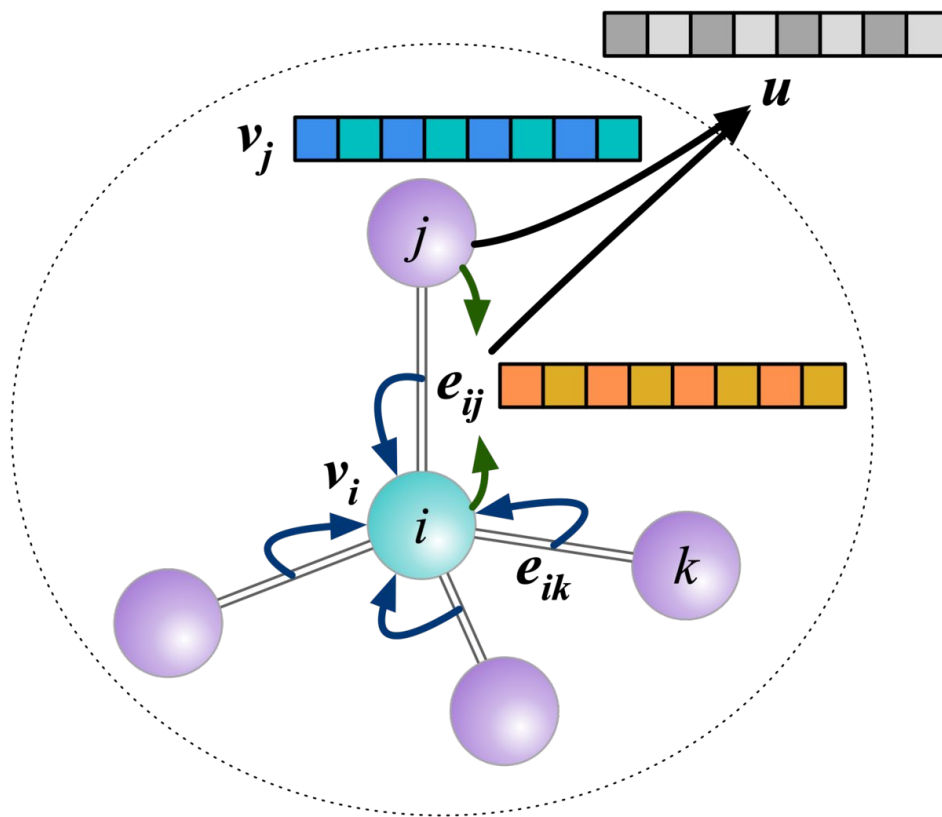


Modification of CGCNN (2)

OI: Ordered intermetallic
CS: core-shell-type alloys



M3GNet: 원자간 퍼텐셜

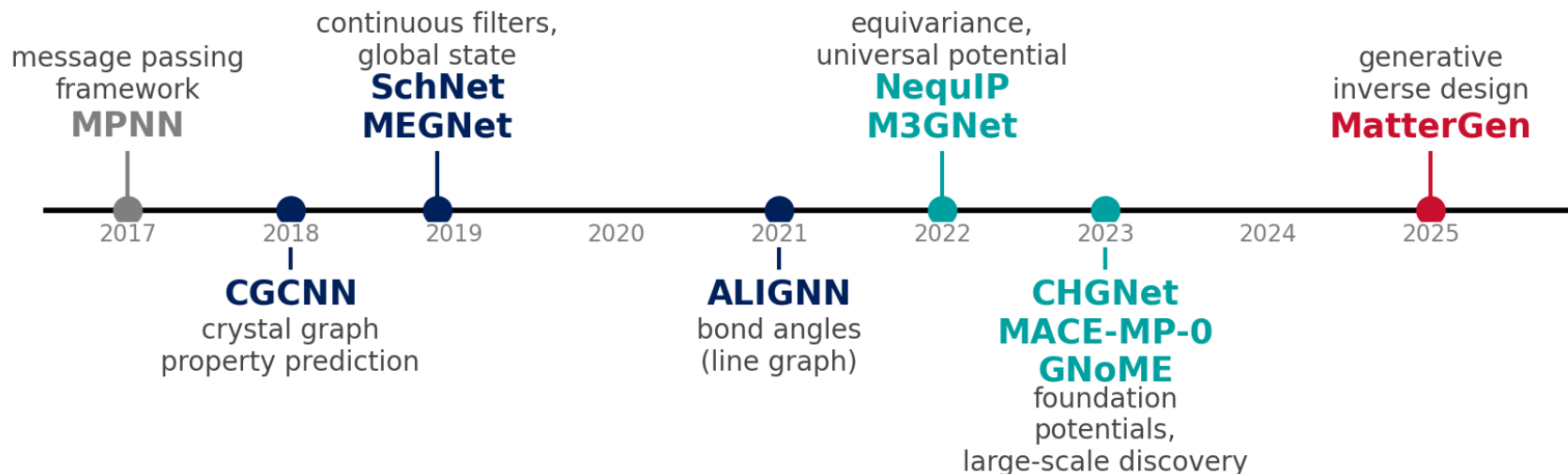


에너지 · 힘 · 응력을 함께 학습

- GNN 출력 = 에너지 E , 힘 $F = -\partial E / \partial r \rightarrow$ 원자간 퍼텐셜 (MLIP)
- 3-body 상호작용(결합각) 포함
- Materials Project의 구조 이완 궤적으로 학습, 주기율표 89개 원소를 하나의 모델로
- DFT 없이 구조 이완 · MD · phonon 계산 가능

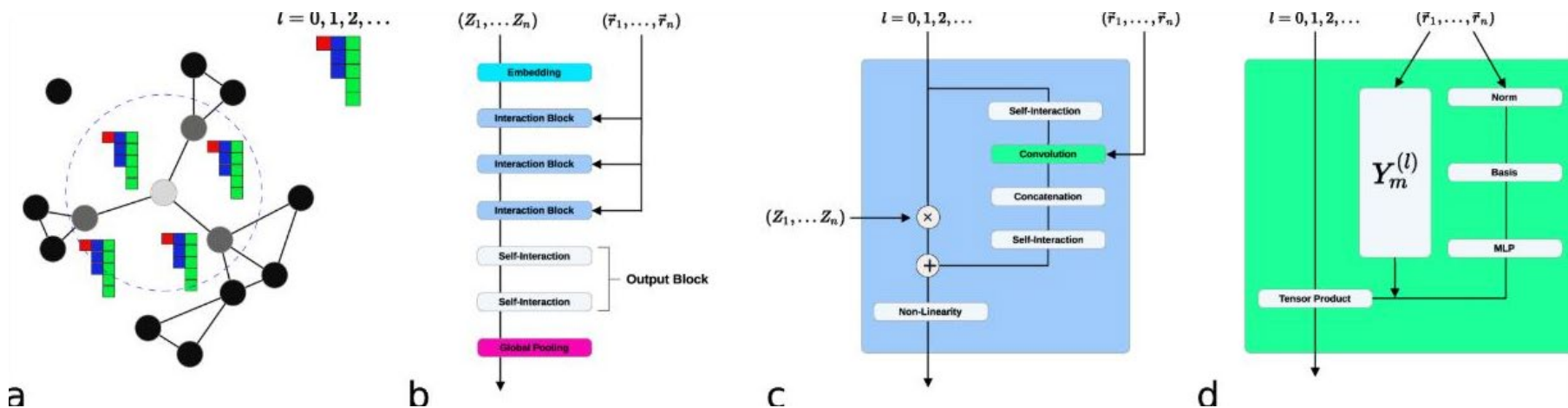
구조만 주면 이완까지 수행 $\rightarrow$ 가상 물질 수천만 개의 안정성 스크리닝에 사용

소재 GNN의 흐름



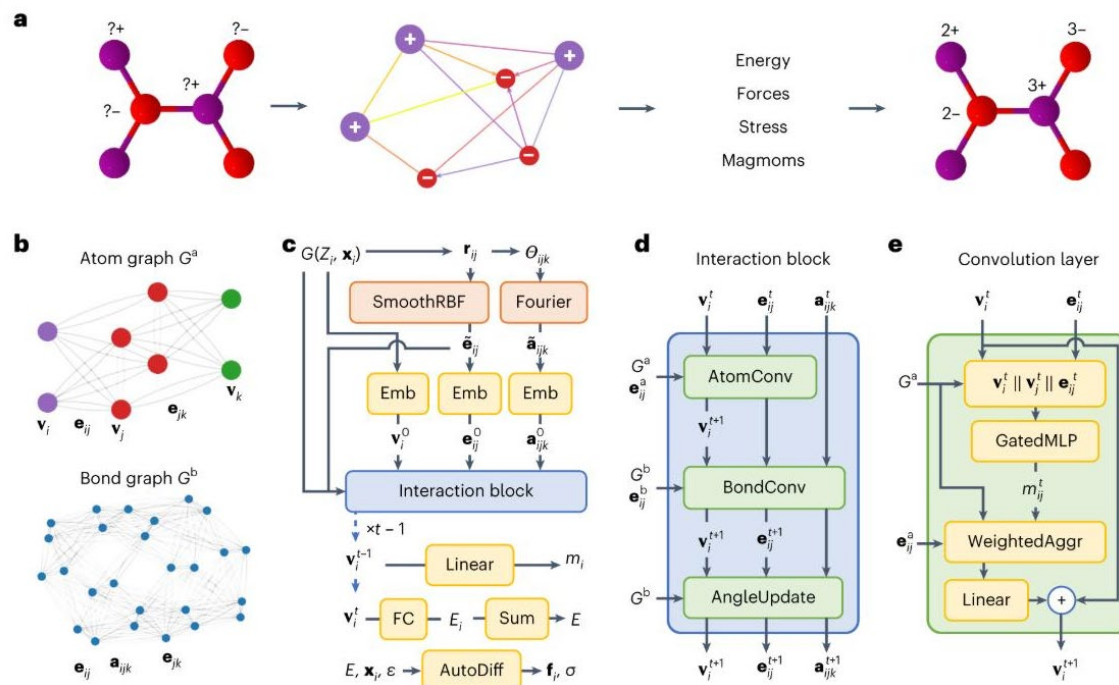
- 불변 → 등변 (equivariant): 거리 · 각도(스칼라)만 쓰던 것에서 벡터 · 텐서 특성으로. NequIP은 적은 데이터로 높은 정확도
- 물성별 모델 → 범용 퍼텐셜: M3GNet, CHGNet, MACE-MP-0. 하나의 사전학습 모델을 fine-tuning해서 사용
- 예측 → 탐색 → 생성: GNoME은 안정한 후보 결정 약 38만 개 제시, MatterGen은 조건에 맞는 구조를 직접 생성

NequIP: 등변 신경망



- **E(3)-equivariant:** 원자 특성이 스칼라만이 아니라 회전에 따라 함께 변하는 텐서 ($l = 0, 1, 2, \dots$)
- 구면조화함수 Y 와 tensor product로 이웃의 방향 정보를 유지한 채 message passing
- 거리 · 각도만 쓰는 불변 모델보다 훨씬 적은 학습 데이터로 높은 정확도 (data efficiency)
- 에너지를 예측하고 힘은 에너지의 미분으로 계산 → 에너지 보존

CHGNet: 전하 정보를 담은 퍼텐셜

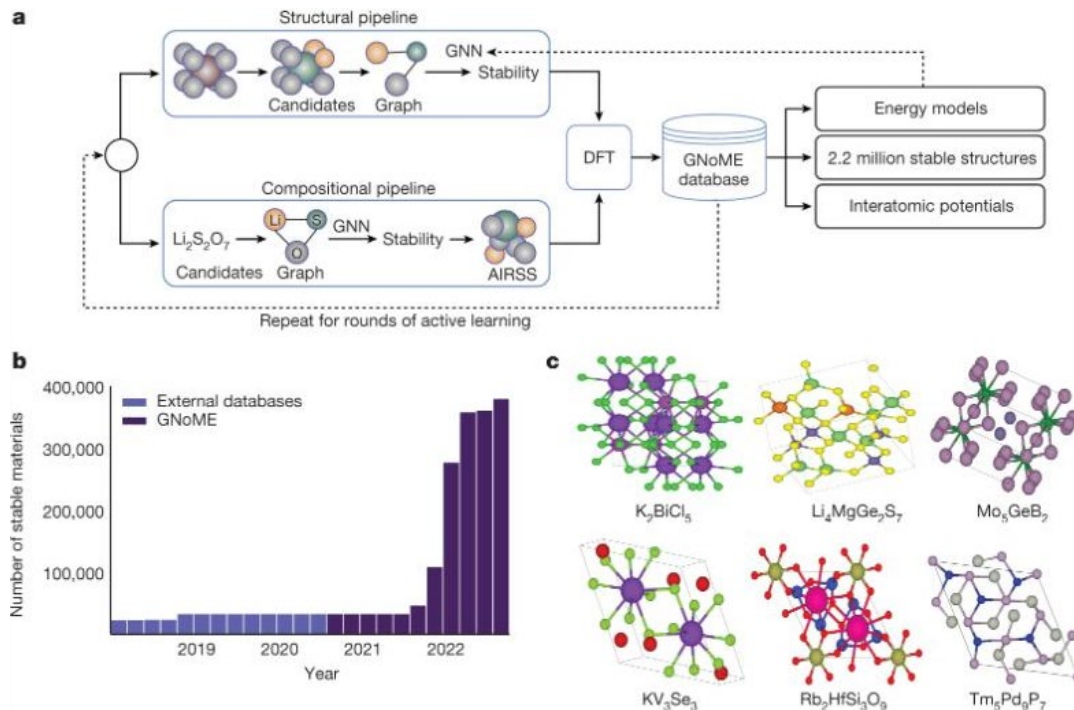


Crystal Hamiltonian GNN

- 에너지 · 힘 · 응력에 더해 자기 모멘트(magmom)를 학습
- 자기 모멘트로 원자가(전하 상태)를 구분
- atom graph와 bond graph(결합각)를 함께 사용

- Materials Project 이완 궤적(MPtrj) 약 158만 구조로 사전학습한 범용 퍼텐셜
- 전이금속 산화물처럼 산화수가 바뀌는 계의 MD에 활용 (예: 배터리 양극재)

GNoME: 대규모 물질 탐색



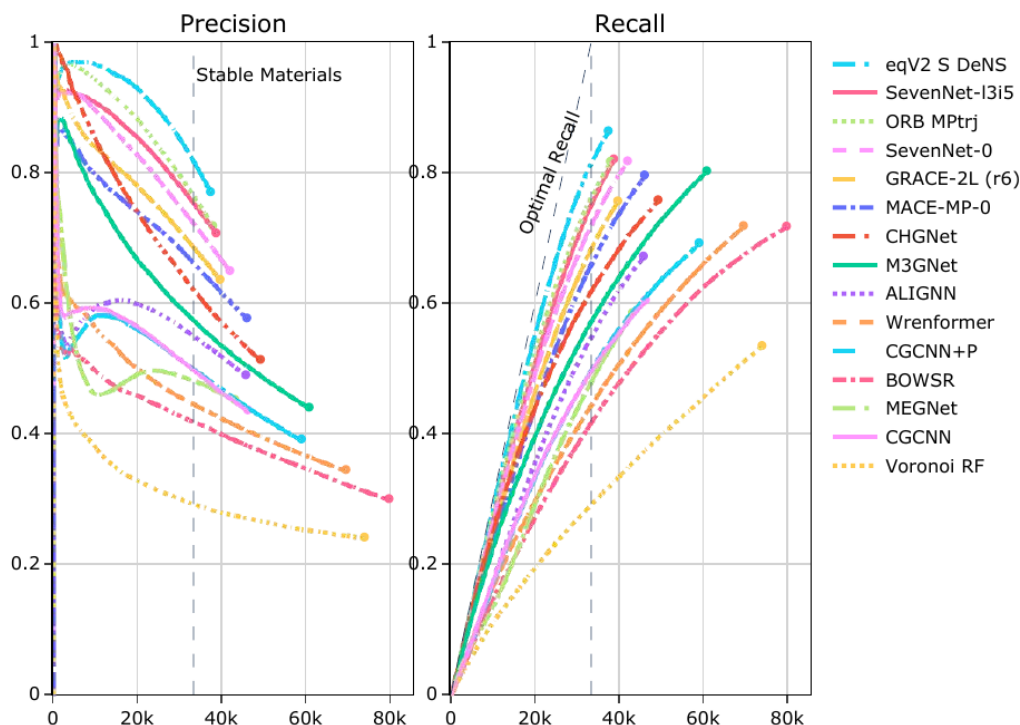
Graph Networks for Materials Exploration

- 후보 구조 생성 → GNN으로 안정성 예측 → DFT 검증 → 재학습 (active learning)
- 새 결정 구조 220만 개, 그중 convex hull 위의 안정 구조 약 38만 개
- 학습 데이터가 늘수록 분포 밖 예측 오차도 감소

예측 모델이 탐색 도구가 된 사례

남는 질문: 예측된 안정 물질을 실제로 합성할 수 있는가 → 자율실험실, 생성 모델로 이어짐

Matbench Discovery (1)



안정한 결정 탐색 벤치마크

- 학습: Materials Project
- 시험: WBM 데이터셋 약 21.5만 구조 (중복 prototype 제거 후, 학습 데이터에 없는 구조)
- 이완 전 구조에서 안정성(convex hull 기준) 예측
- MAE보다 분류 지표 (F1, precision, DAF)가 탐색 성능을 잘 반영
- 범용 원자간 퍼텐셜이 에너지만 직접 예측하는 모델보다 우수

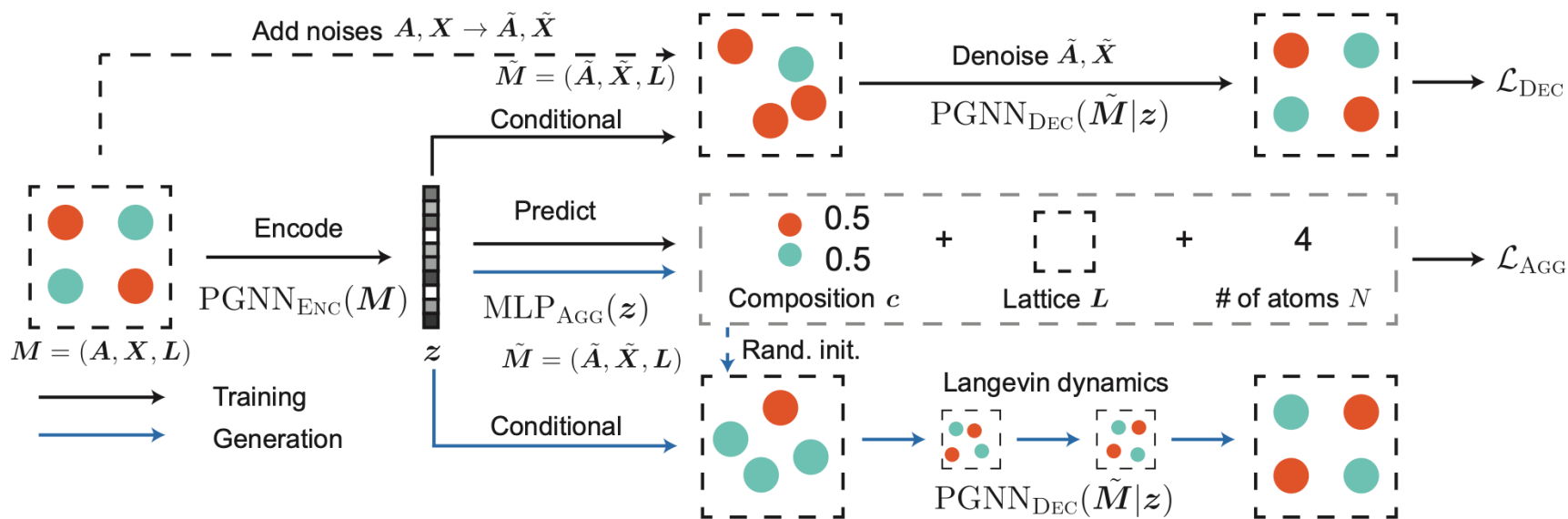
회귀 오차가 작아도 hull 근처에서 틀리면 탐색에는 쓸 수 없음 → 목적에 맞는 평가 지표가 중요

Matbench Discovery (2)

Model	F1 ↑	DAF ↑	Prec ↑	Acc ↑	TPR ↑	TNR ↑	MAE ↓	RMSE ↓	R ² ↑	Training Set	Model Params	Targets	Date Added
eqV2 M	0.917	6.05	0.924	0.975	0.91	0.986	0.02	0.072	0.848	3M (102.4M) (OMat24+MPtrj)	86.6M	EFS _D	2024-10-18
DPA3-v1-OpenLAM	0.883	5.75	0.88	0.963	0.885	0.978	0.023	0.067	0.869	7M (112.9M) (OMat24+MPtrj+sAlex)	8.2M	EFS _G	2025-01-10
GRACE-2L-OAM	0.88	5.77	0.883	0.963	0.878	0.979	0.023	0.068	0.862	7M (112.9M) (OMat24+sAlex+MPtrj)	12.6M	EFS _G	2025-02-06
ORB	0.88	6.04	0.924	0.965	0.841	0.987	0.028	0.077	0.824	3M (32.1M) (MPtrj+Alex)	25.2M	EFS _D	2024-10-11
MatterSim v1 5M	0.862	5.85	0.895	0.959	0.831	0.982	0.024	0.068	0.863	17M (MatterSim)	4.5M	EFS _G	2024-12-16
MACE-MPA-0	0.852	5.58	0.853	0.954	0.851	0.973	0.028	0.073	0.842	3M (12.0M) (MPtrj+sAlex)	9.1M	EFS _G	2024-12-09
GNoME	0.829	5.52	0.844	0.955	0.814	0.972	0.035	0.085	0.785	6M (89.0M) (GNoME)	16.2M	EF _G	2024-02-03
GRACE-1L-OAM	0.824	5.25	0.803	0.944	0.846	0.962	0.031	0.073	0.842	7M (112.9M) (OMat24+sAlex+MPtrj)	3.4M	EFS _G	2025-02-06
eqV2 S DeNS	0.815	5.04	0.771	0.941	0.864	0.953	0.036	0.085	0.788	146K (1.6M) (MPtrj)	31.2M	EFS _D	2024-10-18
DPA3-v1-MPtrj	0.765	4.65	0.711	0.921	0.828	0.938	0.042	0.083	0.798	146K (1.6M) (MPtrj)	3.4M	EFS _G	2025-01-10
ORB MPtrj	0.765	4.7	0.719	0.922	0.817	0.941	0.045	0.091	0.756	146K (1.6M) (MPtrj)	25.2M	EFS _D	2024-10-14
SevenNet-I3I5	0.76	4.63	0.708	0.92	0.821	0.938	0.044	0.087	0.776	146K (1.6M) (MPtrj)	1.2M	EFS _G	2024-12-10
SevenNet-0	0.724	4.25	0.65	0.904	0.818	0.919	0.048	0.092	0.75	146K (1.6M) (MPtrj)	842.4K	EFS _G	2024-07-13
GRACE-2L (r6)	0.691	4.16	0.636	0.896	0.757	0.921	0.052	0.094	0.741	146K (1.6M) (MPtrj)	15.3M	EFS _G	2024-11-21
MACE-MP-0	0.669	3.78	0.577	0.878	0.796	0.893	0.057	0.101	0.697	146K (1.6M) (MPtrj)	4.7M	EFS _G	2023-07-14
CHGNet	0.613	3.36	0.514	0.851	0.758	0.868	0.063	0.103	0.689	146K (1.6M) (MPtrj)	412.5K	EFS _G ^M	2023-03-03
M3GNet	0.569	2.88	0.441	0.813	0.803	0.813	0.075	0.118	0.585	63K (188.3K) (MPE)	227.5K	EFS _G	2022-09-20
ALIGNN	0.567	3.21	0.49	0.841	0.672	0.872	0.093	0.154	0.297	155K (MP 2022)	4.0M	Energy	2023-06-02
MEGNet	0.51	2.96	0.452	0.826	0.585	0.87	0.13	0.206	-0.248	133K (MP Graphs)	167.8K	Energy	2022-11-14
CGCNN	0.507	2.85	0.436	0.818	0.605	0.857	0.138	0.233	-0.603	155K (MP 2022)	128.4K (N=10)	Energy	2022-12-28
CGCNN+P	0.5	2.56	0.392	0.786	0.693	0.803	0.113	0.182	0.019	155K (MP 2022)	128.4K (N=10)	Energy	2023-02-03
Wrenformer	0.466	2.26	0.345	0.745	0.719	0.75	0.11	0.186	-0.018	155K (MP 2022)	5.2M (N=10)	Energy	2022-11-26
BOWSR	0.423	1.96	0.3	0.712	0.718	0.693	0.118	0.167	0.151	133K (MP Graphs)	167.8K	Energy	2022-11-17
Voronoi RF	0.333	1.58	0.241	0.668	0.535	0.692	0.148	0.212	-0.329	155K (MP 2022)	26.2M	Energy	2022-11-26
Dummy	0.185	1	0.154	0.687	0.232	0.769	0.124	0.184	0				

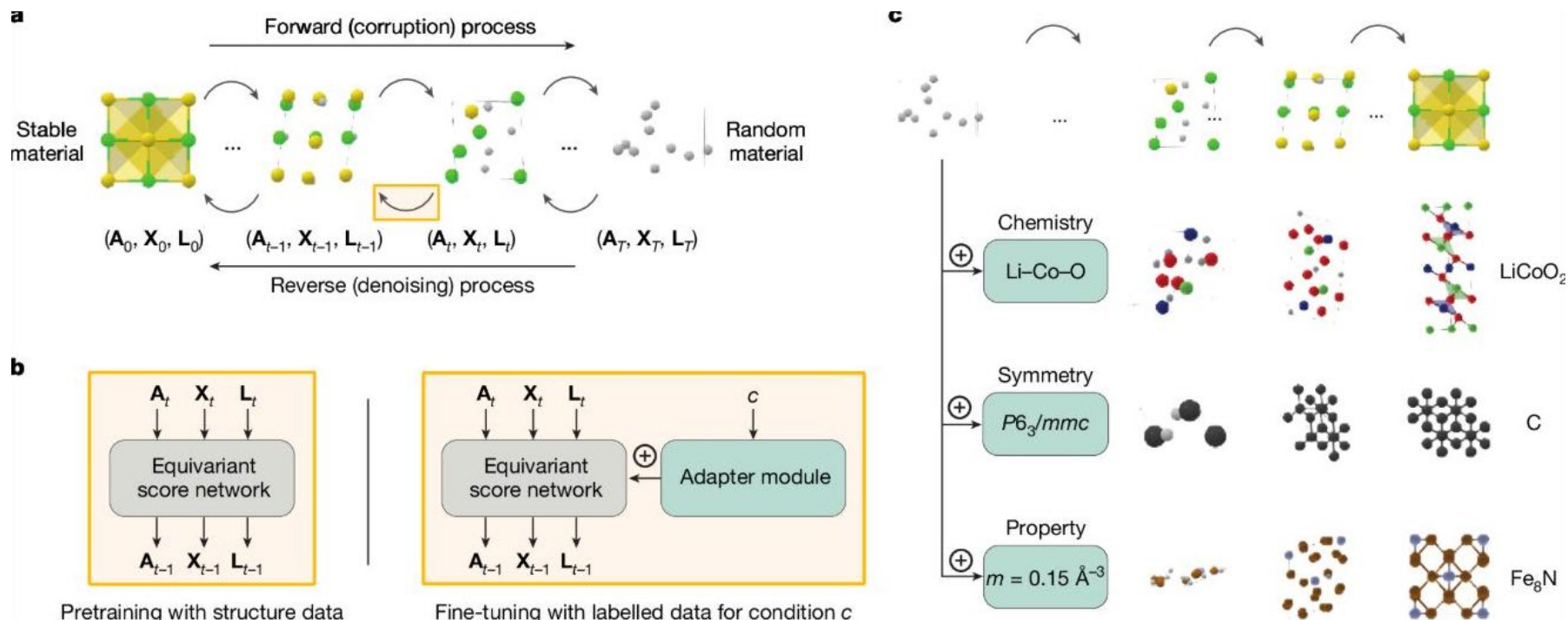
- 아래쪽: CGCNN, MEGNet, ALIGNN (에너지 직접 예측) → 위쪽: M3GNet, CHGNet, MACE 등 범용 퍼텐셜
- 순위는 계속 갱신됨: matbench-discovery.materialsproject.org

생성 모델 (1): CDVAE

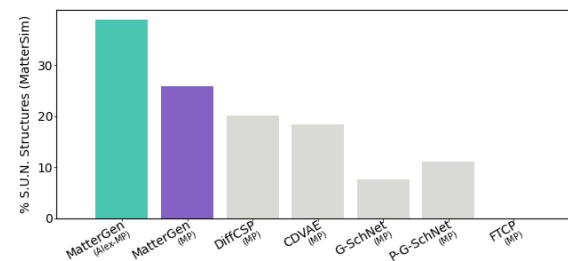


- 예측(구조 $\rightarrow$ 물성)의 반대 방향: 원하는 물성 $\rightarrow$ 구조 생성 (inverse design)
- Encoder(GNN)가 결정을 latent 벡터 z 로 압축, z 에서 조성 · 격자 · 원자 수를 예측
- 원자 좌표에 노이즈를 넣고 되돌리는 denoising(diffusion) 과정으로 안정한 배치를 생성
- 주기성, 회전 · 평행이동 · 순서 불변성을 모델 구조에 반영

생성 모델 (2): MatterGen



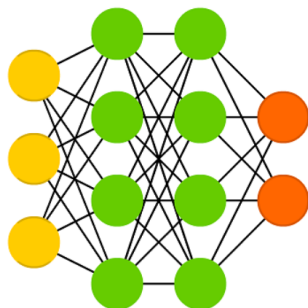
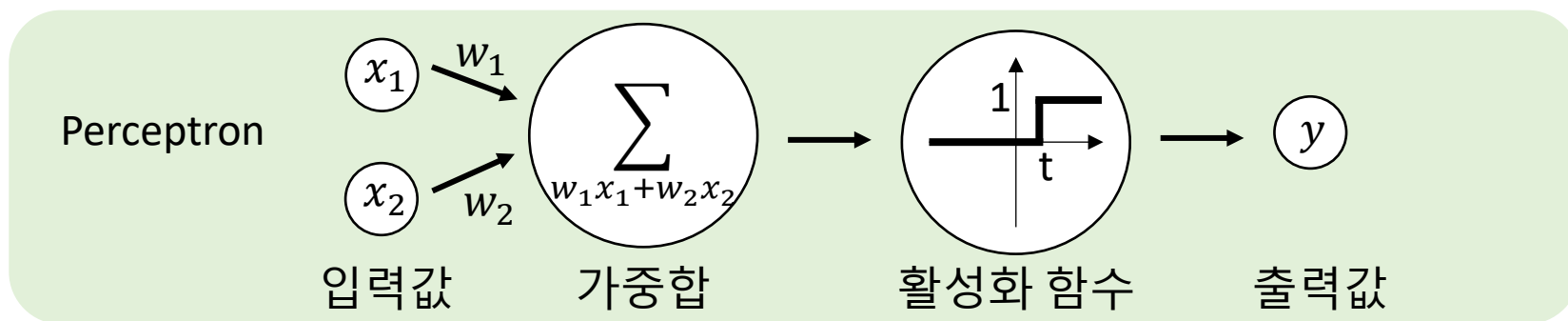
- 원소 종류(A) · 원자 좌표(X) · 격자(L)에 노이즈를 넣고 되돌리는 diffusion 모델
- 화학계, 대칭, 자기 · 전자 · 기계적 물성을 조건으로 fine-tuning (adapter)
- 실험 검증: 체적탄성률 200 GPa 조건으로 생성한 TaCr_2O_6 합성, 측정값 169 GPa



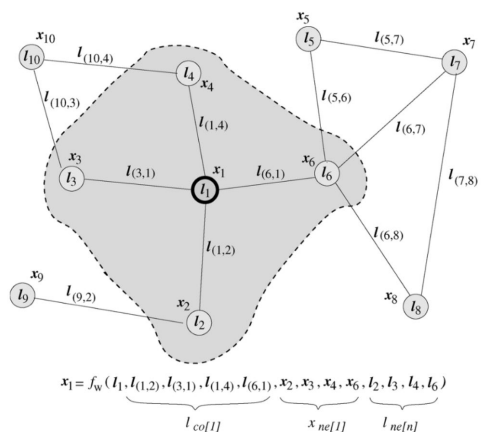
S.U.N. (Stable, Unique, Novel) 비율 비교

Summary

- 딥러닝 개념: 퍼셉트론 → 신경망 → 학습(순전파 · 역전파)과 일반화
- 그래프 기반 소재 딥러닝: GNN/CGCNN → 범용 퍼텐셜 → 생성 모델

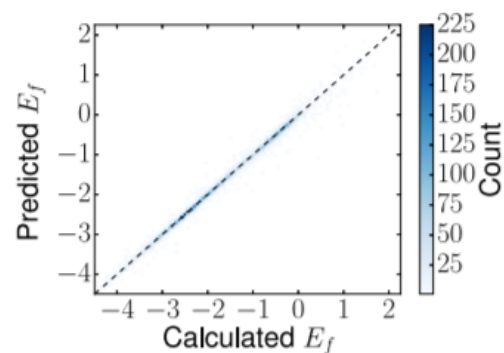


Deep feedforward network (DFN)



An illustration of node state update based on the information in its neighbors. Figure from "The Graph Neural Network Model".

Graph neural network (GNN)



Predicting materials properties

References

- [교재] 장철원, 선형대수와 통계학으로 배우는 머신러닝 with 파이썬 (2021)
- Choudhary et al., npj Comput. Mater. 8, 59 (2022): 소재 딥러닝 리뷰
- Kingma & Ba, ICLR (2015): Adam
- Srivastava et al., JMLR 15, 1929 (2014): Dropout
- Ioffe & Szegedy, ICML (2015): Batch normalization
- Dunn et al., npj Comput. Mater. 6, 138 (2020): Matbench
- Kwoen & Arakawa, Cryst. Growth Des. 20, 5289 (2020): RHEED CNN
- Scarselli et al., IEEE Trans. Neural Netw. 20, 61 (2009): GNN
- Zhou et al., AI Open 1, 57 (2020): GNN 리뷰
- Kipf & Welling, ICLR (2017): GCN
- Gilmer et al., ICML (2017): MPNN
- Sanchez-Lengeling et al., Distill (2021): GNN 입문
- Xie & Grossman, PRL 120, 145301 (2018): CGCNN
- Chen et al., Chem. Mater. 31, 3564 (2019): MEGNet
- Kim et al., Chem. Mater. 32, 709 (2020): SGCNN
- Choudhary & DeCost, npj Comput. Mater. 7, 185 (2021): ALIGNN
- Batzner et al., Nat. Commun. 13, 2453 (2022): NequIP
- Chen & Ong, Nat. Comput. Sci. 2, 718 (2022): M3GNet
- Deng et al., Nat. Mach. Intell. 5, 1031 (2023): CHGNet
- Merchant et al., Nature 624, 80 (2023): GNoME
- Batatia et al., J. Chem. Phys. 163, 184110 (2025): MACE-MP-0
- Riebesell et al., Nat. Mach. Intell. 7, 836 (2025): Matbench Discovery
- Xie et al., ICLR (2022): CDVAE
- Zeni et al., Nature 639, 624 (2025): MatterGen